

Técnicas de búsqueda y heurísticas

Diplomado en Inteligencia Artificial

Junio 2025

Profesor: Ricardo Contreras A.

En esta clase

- Constraint Programming
- Búsqueda informada y con oponentes
 - BF, A*
 - Minimax
 - Markov
 - Optimalidad de Bellman*

Constraint Programming

Constraint programming

- Surge en los 1980s. Es un paradigma derivado desde el mundo la IA, la investigación de operaciones y la algoritmia
 - Prolog III (Marseille, Francia)
 - CLP(R)
 - CHIP (ECRC, Alemania)
- Se usa para resolver problemas de optimización combnatoria

- Dos contribuciones principales
 - Una nueva mirada a la optimización combinatoria
 - Complementa los métodos standard de OR
 - Combinatoria versus numérica
 - Un lenguaje nuevo para optimización combinatoria
 - Un lenguaje potente para restricciones
 - Lenguaje para procedimientos de búsqueda
 - Extensiones

Ejemplo muy sencillo

- Colorear un mapa de (parte de) Europa: Bélgica, Dinamarca, Francia, Alemania, Países Bajos, Luxemburgo
- No puede haber dos países adyacentes con el mismo color
- ¿Son suficientes cuatro colores?

El lenguaje opl

- Optimization Programming Language es un lenguaje de modelado algebraico para optimización matemática.
- Fácil codificación
- Es parte del software CPLEX

Ejemplo en OPL

```
enum Country
{Belgica,Dinamarca,Francia,Alemania,PaísesBajos,Luxemburgo};

enum Colors {azul,rojo, amarillo,gris};

var Colors color[Country];

solve {
    color[Francia] <> color[Belgica];
    color[Francia] <> color[Luxemburgo];
    color[Francia] <> color[Alemania];
    color[Luxemburgo] <> color[Alemania];
    color[Luxemburgo] <> color[Belgica];
    color[Belgica] <> color[PaísesBajos];
    color[Belgica] <> color[Alemania];
    color[Alemania] <> color[PaísesBajos];
    color[Alemania] <> color[Dinamarca];
};
```

- Variables no-numéricas
- Restricciones no-lineales
- Perfectamente legal en CP

Facilidad de Formulación

- Requerimientos lógicos: if $A = 1$ and $B \leq 2$ then either $C \geq 3$ or $D=1$.
- Directo en CP:

$$((A=1) \ \& \ (B \leq 2)) \Rightarrow ((C \geq 3) \vee (D=1))$$

Restricción Global: alldifferent

- Los más conocidos son restricciones ya estudiadas

`alldifferent(x, y, z)`

Indica que x , y , y z toman diferentes valores. Así, puede darse que $x=2$, $y=1$, $z=3$, pero no $x=1$, $y=3$, $z=1$.

Esto es muy usado en ruteamiento ($x[i]$ es el i -ésimo cliente visitado, `alldifferent[x]` nos indica que cada cliente es visitado a lo más una vez), y muy útil en otras muchas situaciones.

Restricciones Globales

- Se pueden crear cuando se requiera
 - Potencian y expanden el lenguaje
 - Facilitan el modelamiento y el lenguaje es más natural
 - Encuentra rápidamente soluciones
 - Detalles son transparentes al usuario

Scheduling

- Maneja conceptos de “jobs”, tareas, “antes”, “después”. Los “jobs” necesitan máquinas, entre otras situaciones
- Fácil de extender

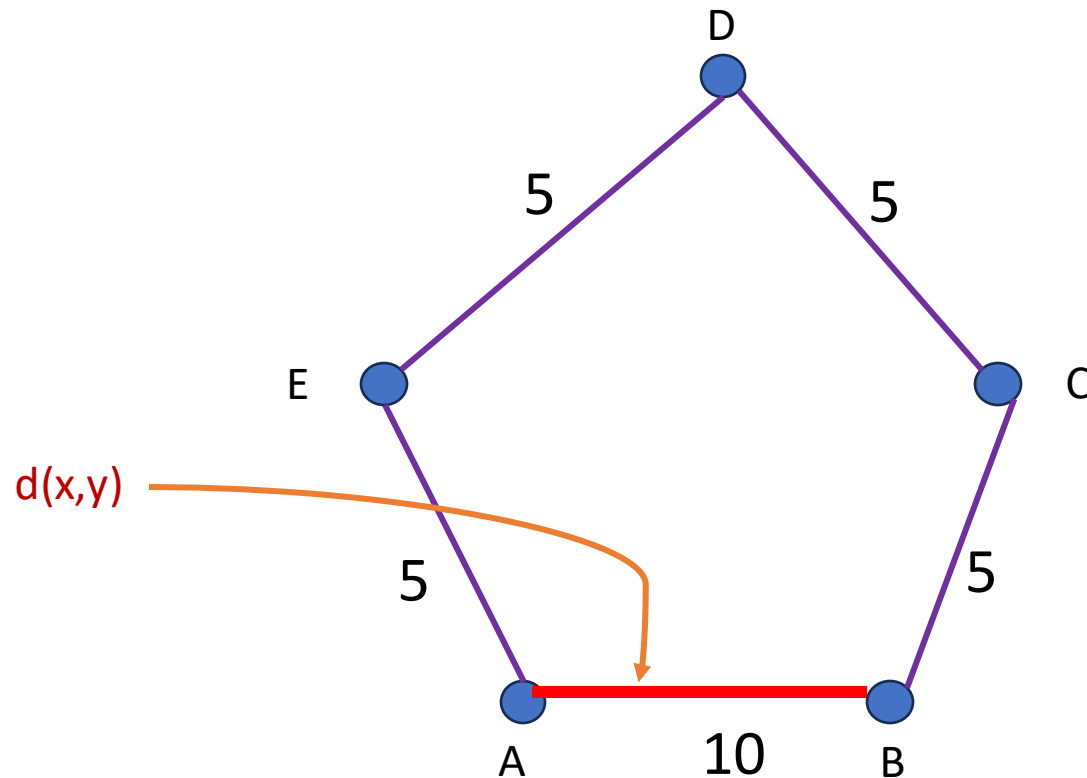
Volvamos a
los grafos



Los Héroes solo combinación

Definición

- La **distancia** entre los vértices **x** e **y** de un grafo conexo **G** es la longitud de la trayectoria más corta desde **x** a **y** y se denota como **$d(x,y)$**



Definición

- Sea P una propiedad cualquiera de los grafos (e.g. # de vértices, # de aristas), entonces el grafo G , si G posee la propiedad P , se dice **minimal** con respecto a P ssi para cada arista $e \in E(G)$ (o respecto a cada vértice $v \in V(G)$), $G-e$ (o $G-v$) no posee la propiedad P .

Más claro...

- Por ejemplo, si G es un grafo, si eliminamos cualquier arista (o cualquier vértice) desde G , y G pierde la propiedad, entonces decimos que G es minimal con respecto a esa propiedad
- ¿Cuál es el minimal para un grafo conexo?

Definiciones

- Un grafo es **acíclico** si no contiene ciclos.
- Un grafo es un **árbol** ssi es conexo y acíclico.
- Por lo tanto es claro que los grafos conexos minimales son árboles.

Arboles de cobertura mínimos (MST)

- Sea $G=(V,E)$ un grafo conexo con V vértices y E aristas, y un peso W_{ij} para cada arista $ij \in E$.
- Un árbol de cobertura mínimo (MST) es un árbol de cobertura con suma mínima de los pesos de sus aristas.
- El peso de una arista puede representar muchas cosas. Si representa el costo de comunicación de un mensaje a través de la arista en cualquier dirección, el peso total representa el costo de transmitir un mensaje que llegue a todos los nodos del árbol de cobertura.

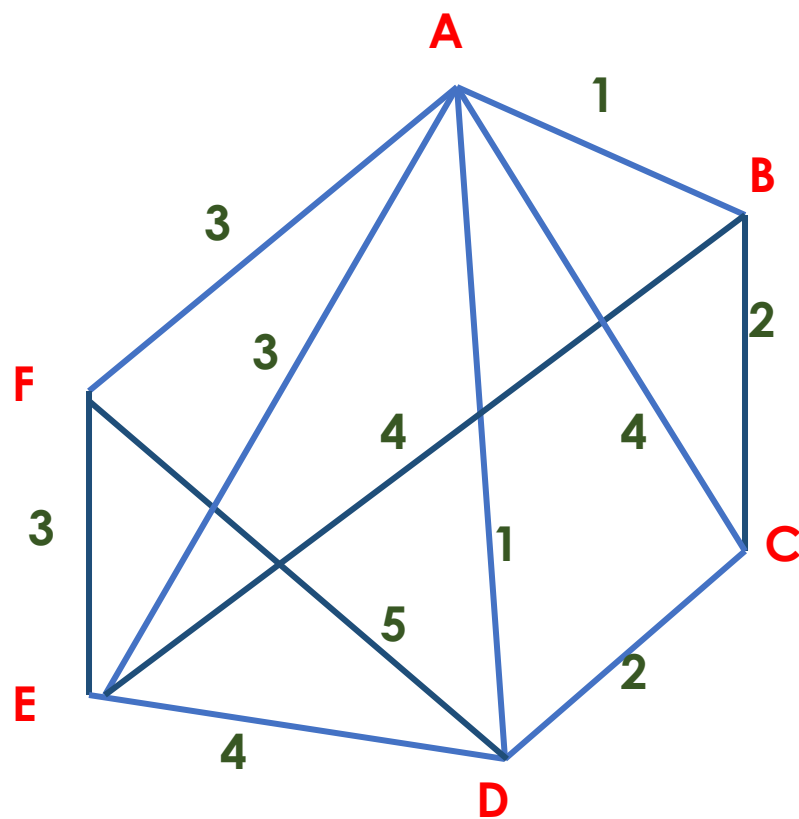
Un algoritmo para construir MST

- **Algoritmo de Prim.** Se inicia seleccionando un vértice arbitrario (con lo que tenemos un **fragmento**). Luego, se aumenta el **fragmento** por adición sucesiva de una arista de salida de peso mínimo.
- Este algoritmo termina en $n-1$ iteraciones, en que n es el número de vértices en el grafo.

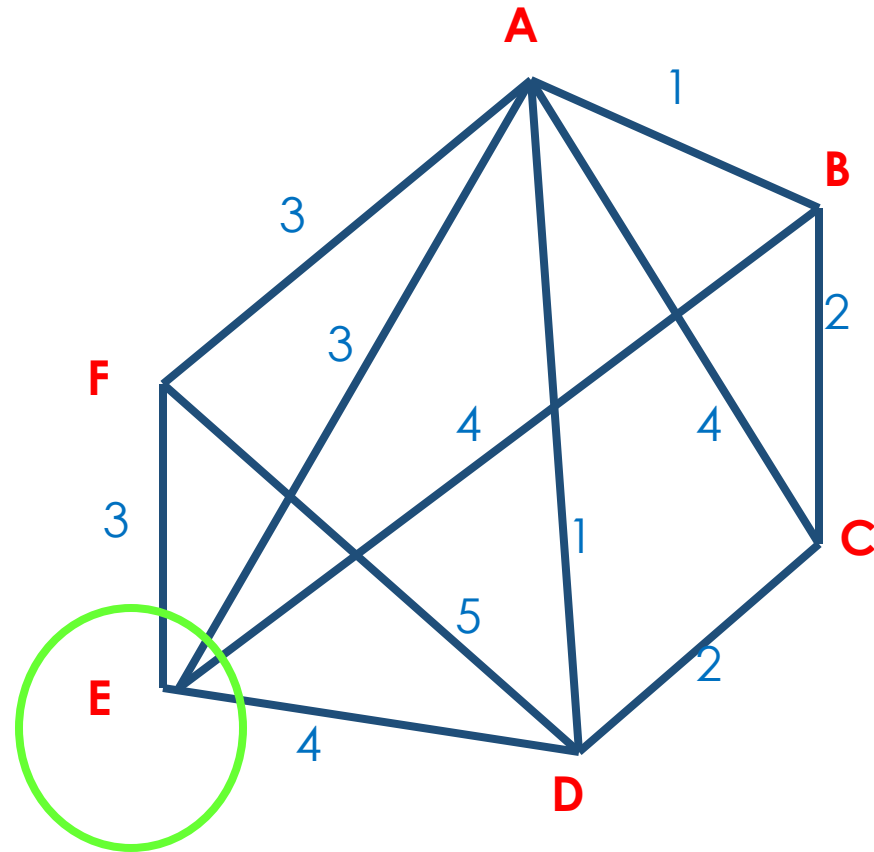
Algoritmo de Prim

- 1. Escoja cualquier vértice v_1 de G .
- 2. Escoja una arista $e_1 = v_1 v_2$ de G tal que $v_2 \neq v_1$ y e_1 tiene el menor peso entre las aristas de G incidentes en v_1 .
- 3. Si las aristas $e_1, e_2, \dots, e_i$ han sido escogidas involucrando puntos terminales $v_1, \dots, v_{i+1}$; escoja una arista $e_{i+1} = v_j v_k$ con $v_j \notin \{v_1, \dots, v_{i+1}\}$ y $v_k \notin \{v_1, \dots, v_{i+1}\}$ tal que e_{i+1} tiene el menor peso entre las aristas de G con exactamente un extremo en $\{v_1, \dots, v_{i+1}\}$.
- 4. Deténgase cuando se hayan escogido $n-1$ aristas. Si no, repetir el paso 3.

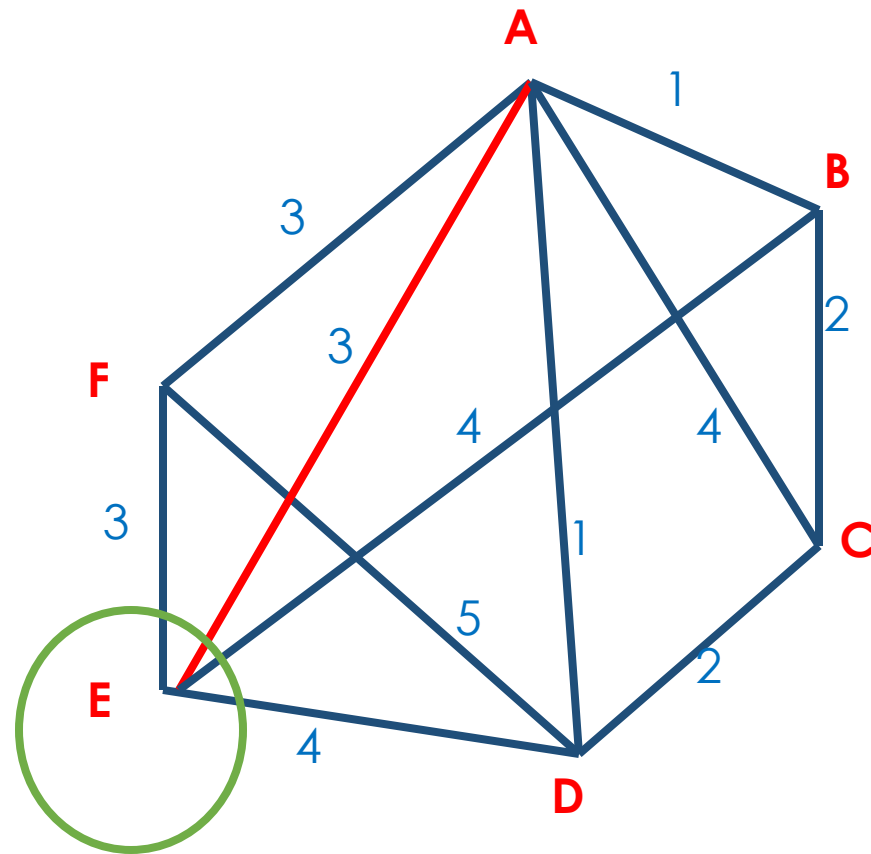
Ejemplo



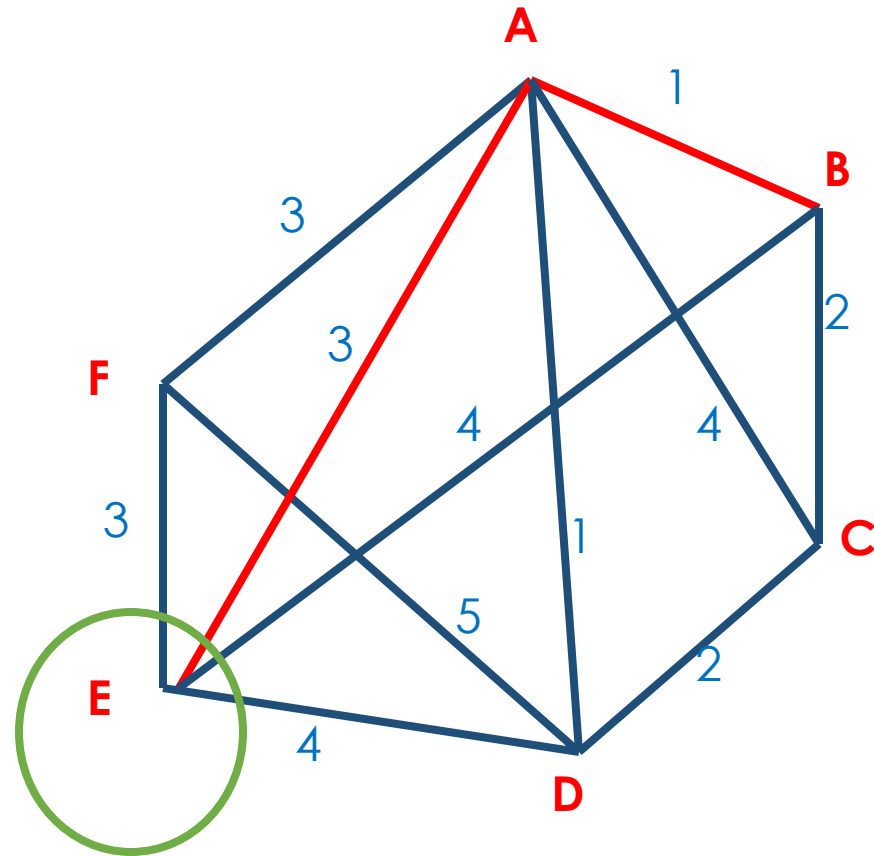
Elijamos arbitrariamente un vértice: ... E



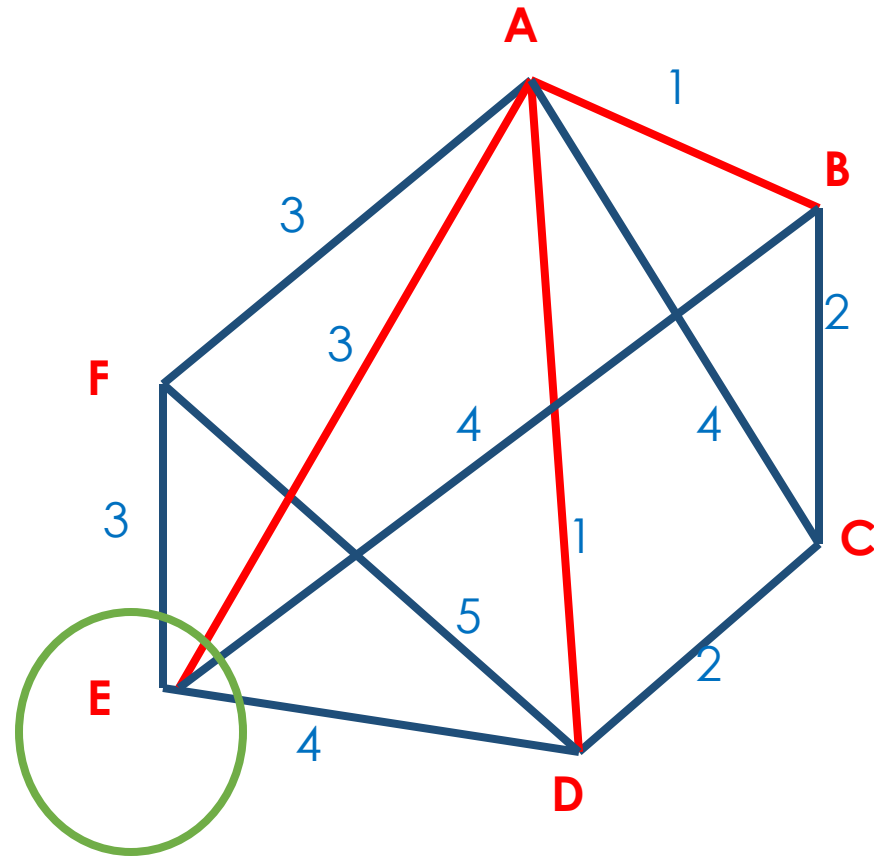
De las dos aristas salientes con igual peso mínimo, escogemos arbitrariamente una...



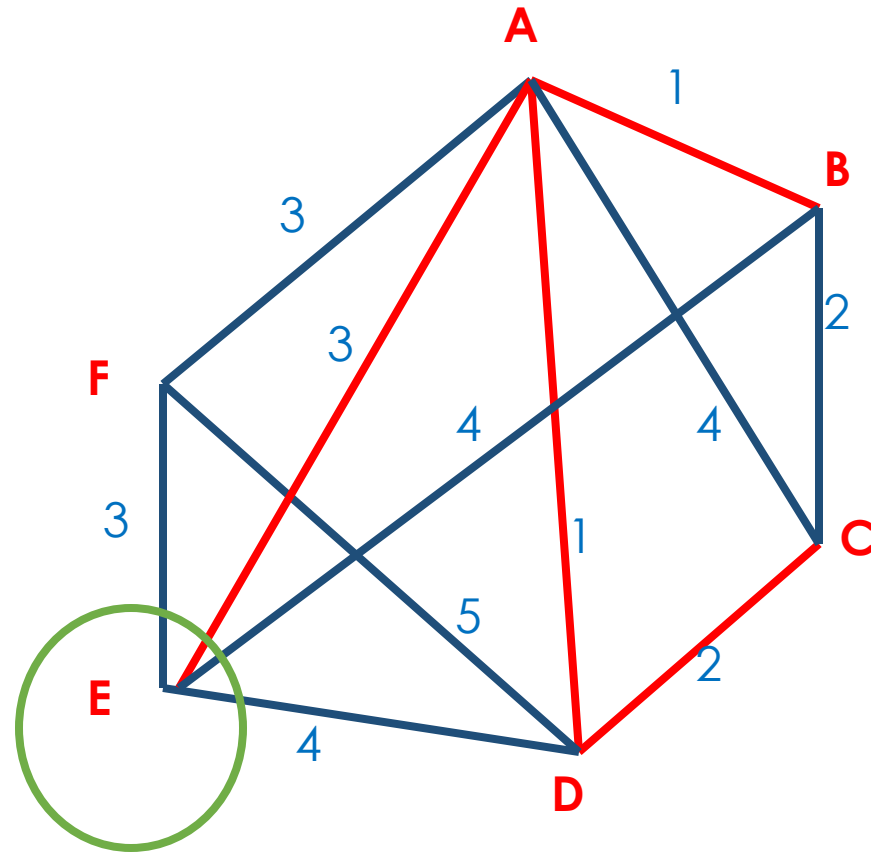
De las dos aristas salientes con igual peso mínimo, escogemos arbitrariamente una...



Y repetimos el proceso ya que todavía no tenemos las $n-1$ aristas.



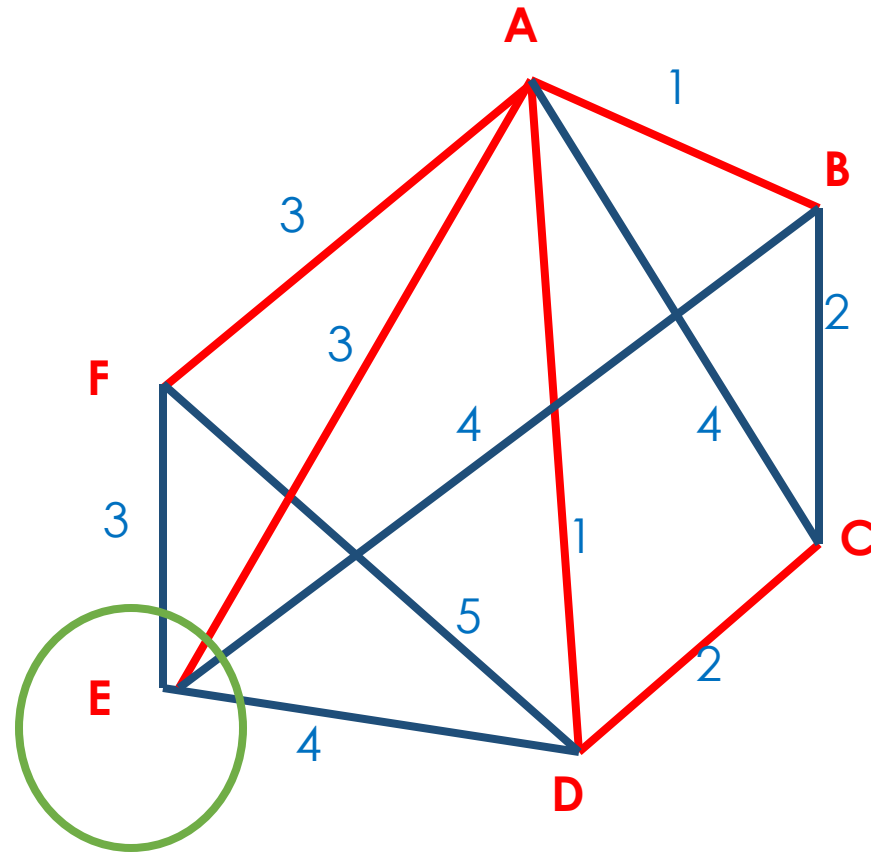
Ahora tenemos dos aristas elegibles, de peso 2.



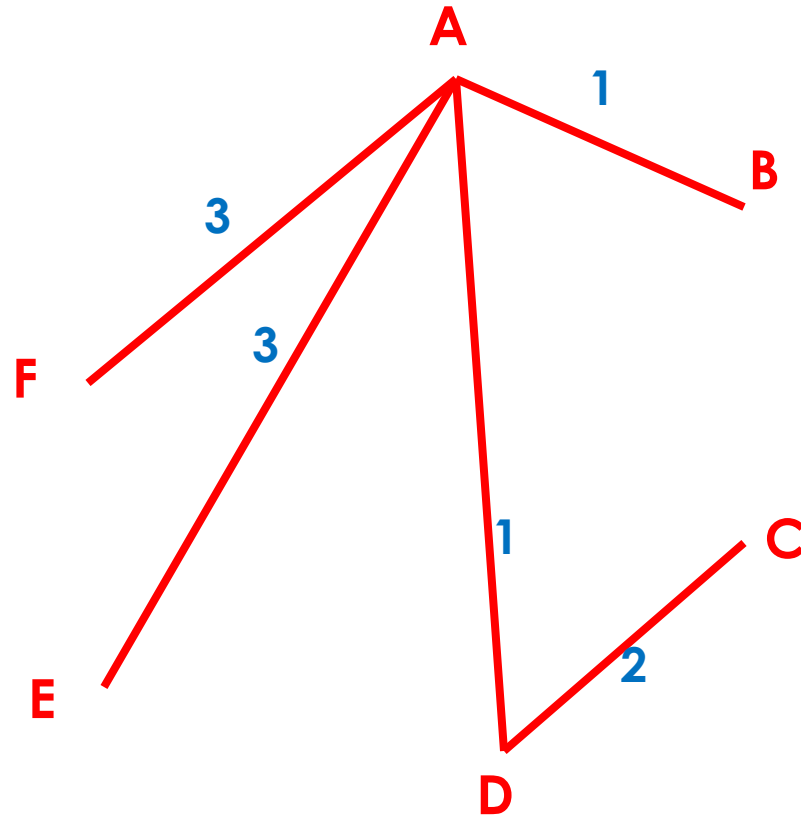
Elegimos arbitrariamente

... y para incluir el vértice F, entre las dos opciones

Elegimos arbitrariamente



En consecuencia, el árbol de cobertura mínimo construido es

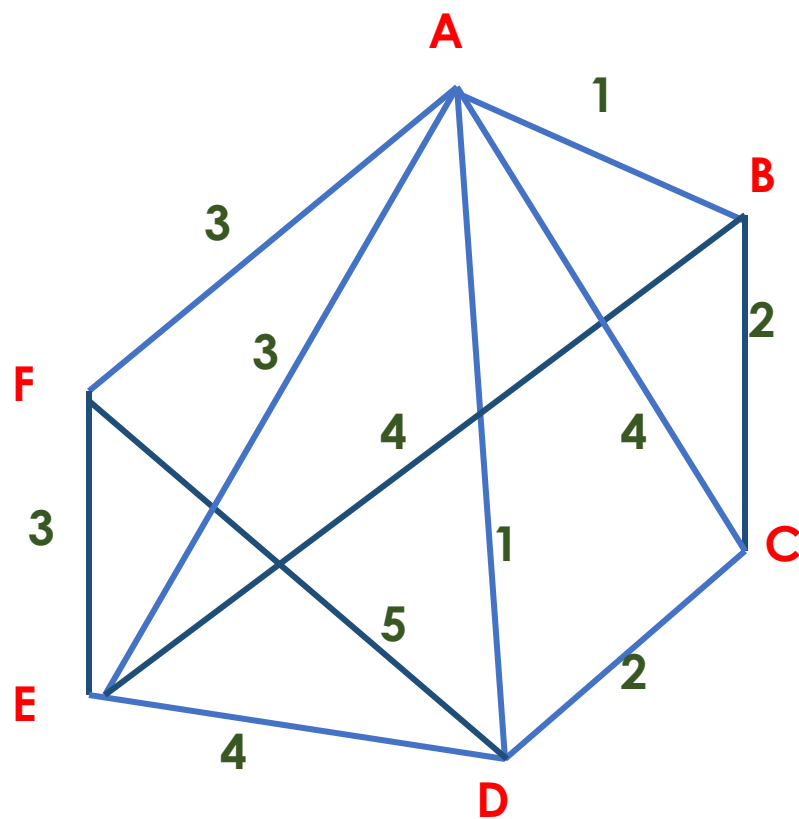


y su peso es 10

¿Y si nuestras elecciones arbitrarias
hubieran sido otras?

- ¿Es evidente que el árbol sería diferente?

Es el mismo grafo anterior



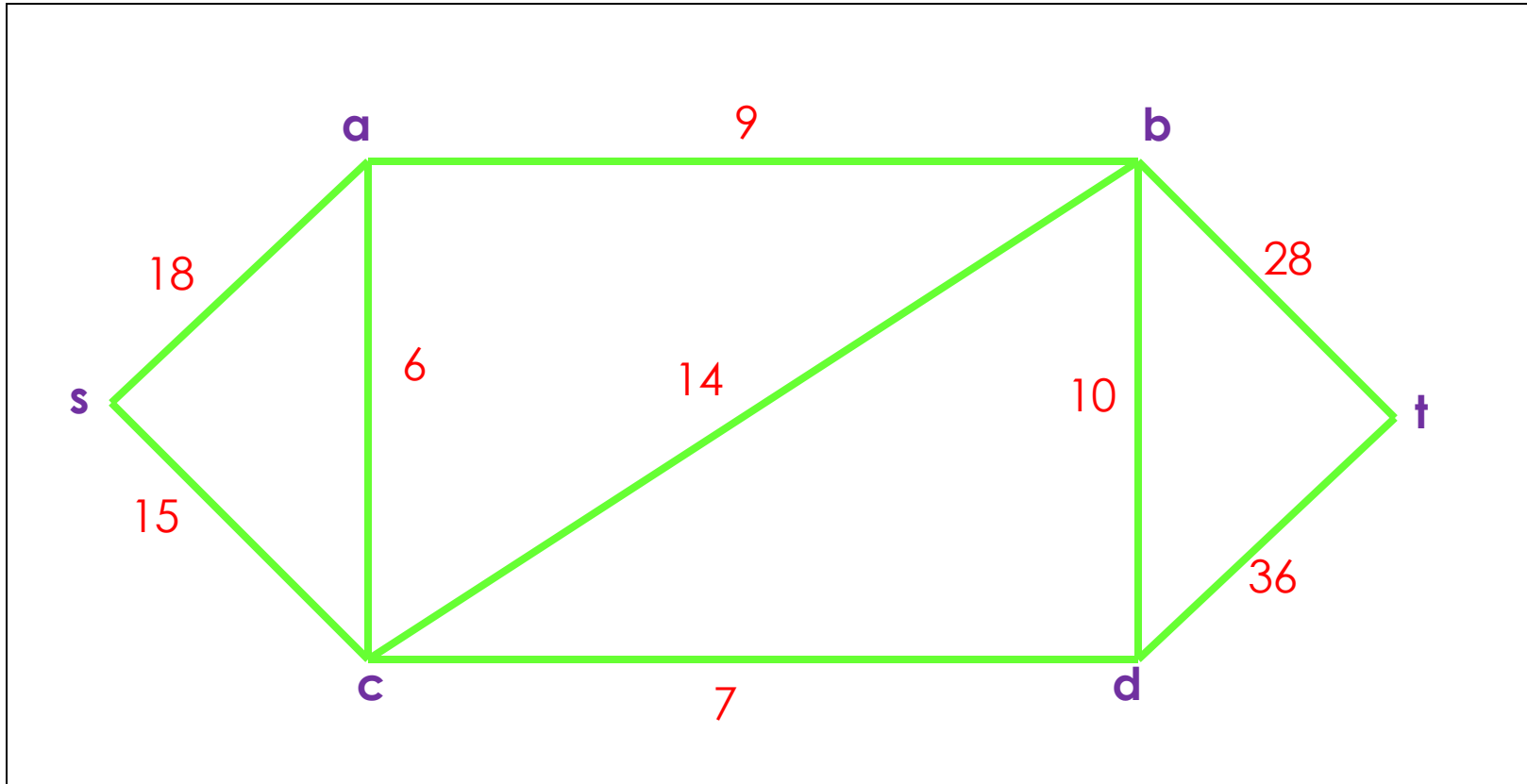
Algoritmo de Dijkstra para el camino mínimo

- Definición.
 - s es el punto de partida
 - t es el punto de término
 - $\lambda(v)$ es la suma de los pesos desde el punto s al punto v
 - V es el conjunto de todos los vértices

Algoritmo de Dijkstra

- Paso 1. Haga $\lambda(s)=0$, y para todos los vértices v distintos de s , haga $\lambda(v)=\infty$. Haga $T=V$.
- Paso 2. Sea u un vértice en T para el cual $\lambda(u)$ es mínimo.
- Paso 3. Si $u = t$, parar.
- Paso 4. Para cada arista $e=uv$ incidente con u , si $v \in T$ y $\lambda(v) > \lambda(u) + w(e)$, cambie el valor de $\lambda(v)$ por $\lambda(u) + w(e)$.
- Paso 5. Sea $T = T - \{u\}$ y vaya al paso 2.

Ejemplo



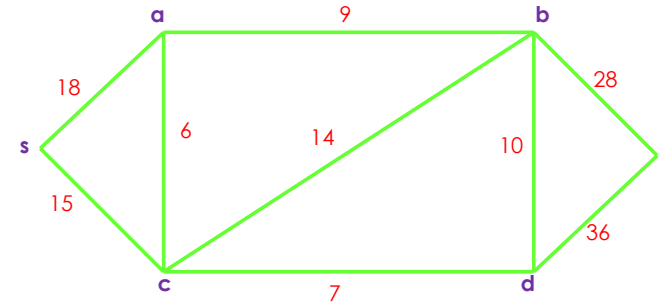
Solución

► Paso 1. Inicialización

Vértice v	s	a	b	c	d	t
$\lambda(v)$	0	∞	∞	∞	∞	∞
T	$\{s, a, b, c, d, t\}$					

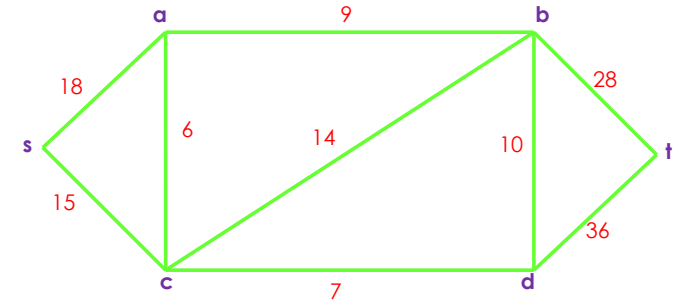
► Paso 2. $u=s$ tiene $\lambda(u)$ mínimo (valor 0).

Solución (cont.)



- Paso 4. Hay dos aristas incidentes en u , sean sa y sc .
 - Ambos vértices, a y c , están en T , es decir, todavía no tienen valor.
 - $\lambda(a) = \infty > 18 = 0 + 18 = \lambda(s) + w(sa)$ así es que $\lambda(a)$ toma el valor 18.
 - De forma similar, $\lambda(c)$ toma el valor 15.

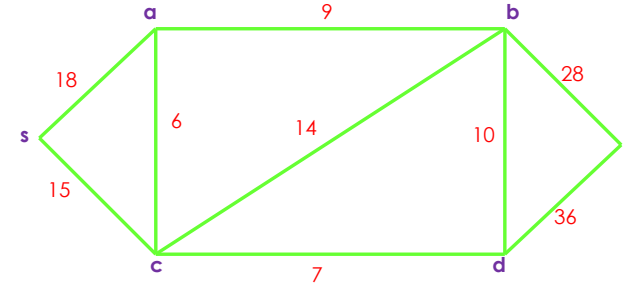
Solución (cont.)



- Paso 5. $T = T - \{s\}$. Así tenemos

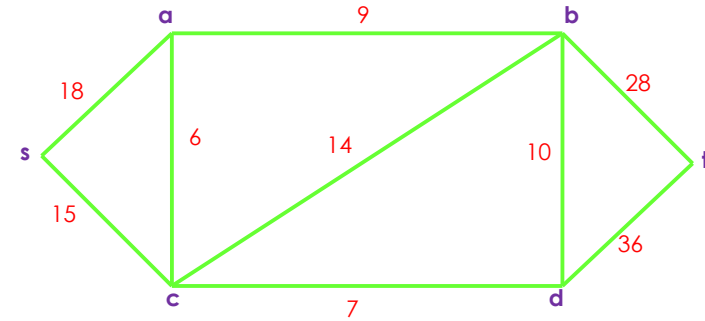
Vértice v	s	a	b	c	d	t
$\lambda(v)$	0	18	∞	15	∞	∞
T	$\{a, b, c, d, t\}$					

Solución (cont.)



- Paso 2. Sea $u=c$ el valor de $\lambda(u)$ mínimo y u en T (con $\lambda(u)=15$).
- Paso 4. Hay tres aristas cv incidentes en $u=c$ que tienen v en T , sean ca , cb , cd .
- $\lambda(a)=18 < 21 = 15 + 6 = \lambda(c)+w(ca)$ por lo que $\lambda(a)$ se mantiene con valor 18.
- $\lambda(b) = \infty > 29 = 15 + 14 = \lambda(c)+w(cb)$ por lo que $\lambda(b)$ toma el valor 29. De manera similar, $\lambda(d)$ es $15 + 7 = 22$.

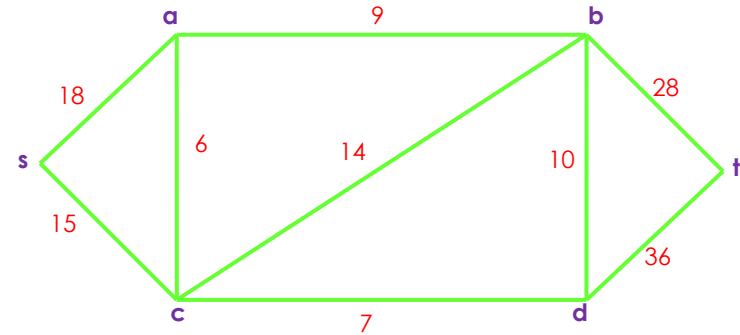
Solución (cont.)



- Paso 5. $T = T - \{c\}$. Así, ahora tenemos

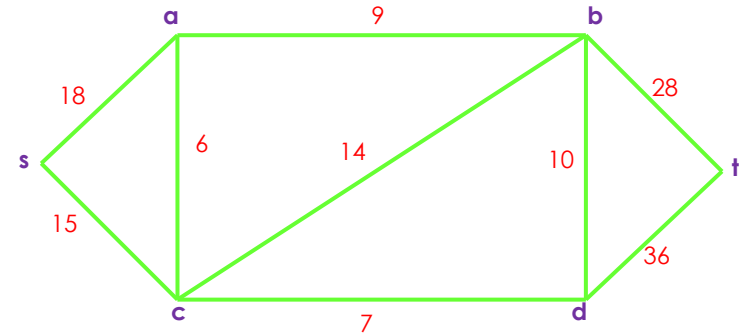
Vértice v	s	a	b	c	d	t
$\lambda(v)$	0	18	29	15	22	∞
T	$\{a, b, d, t\}$					

Solución (cont.)



- Paso 2. $u=a$ tiene $\lambda(u)$ un mínimo para u en T (con $\lambda(u) = 18$).
- Paso 4. Hay sólo una arista av incidente en $u=a$ con v en T , sea ab .
 - $\lambda(b) = 29 > 27 = 18 + 9 = \lambda(a) + w(ab)$ por lo que $\lambda(b)$ toma el valor 27.

Solución (cont.)

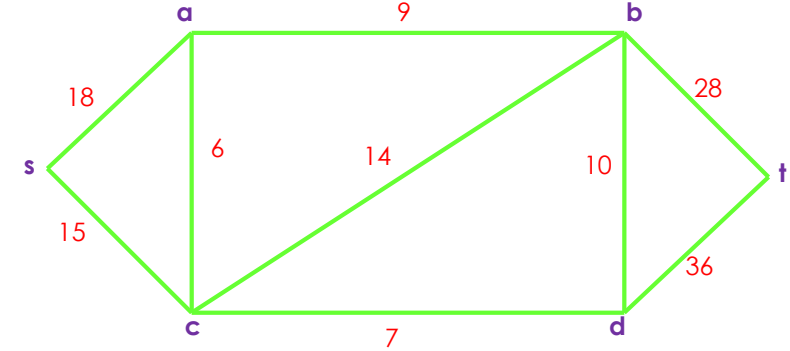


► Paso 5. T es ahora $T - \{a\}$. Así tenemos

Vértice v	s	a	b	c	d	t
$\lambda(v)$	0	18	27	15	22	∞
T	$\{b, d, t\}$					

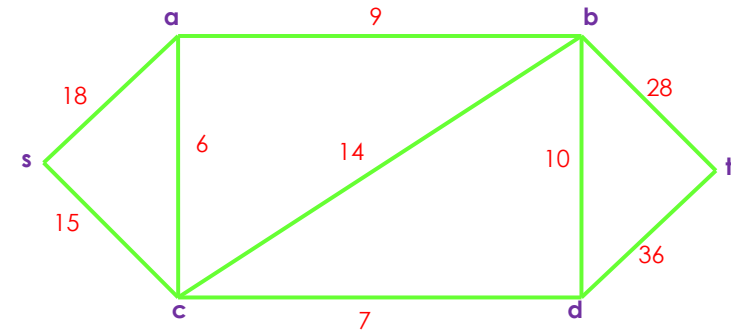
► Paso 2. $u=d$ tiene $\lambda(u)$ mínimo para u en T con $\lambda(u) = 22$.

Solución (cont.)



- Paso 4. Hay dos aristas dv incidentes en $u=d$ que tienen v en T , sean db y dt .
 - $\lambda(b) = 27 < 32 = 22 + 10 = \lambda(d) + w(db)$ por lo que $\lambda(b)$ permanece con valor 27.
 - $\lambda(t) = \infty > 58 = 22 + 36 = \lambda(d) + w(dt)$ por lo que $\lambda(t)$ asume el valor 58.

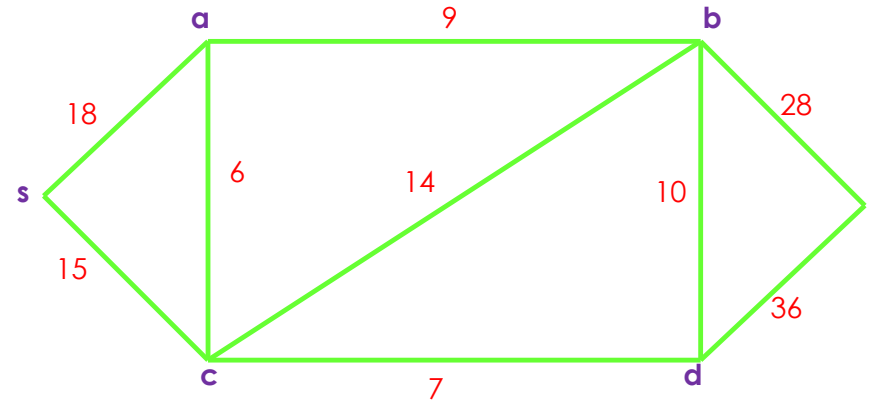
Solución (cont.)



- Paso 5. $T = T - \{d\}$. Así, ahora tenemos

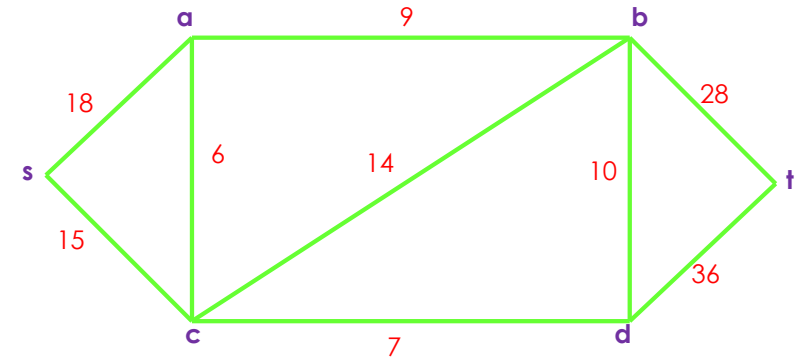
Vértice v	s	a	b	c	d	t
$\lambda(v)$	0	18	27	15	22	58
T	$\{b, t\}$					

Solución (cont.)



- Paso 2. $u=b$ tiene $\lambda(u)$ mínimo para u en T con $\lambda(u) = 27$.
- Paso 4. Hay sólo una arista bv para v en T ,
 - sea bt $\lambda(t) = 58 > 55 = 27 + 28 = \lambda(b) + w(bt)$ por lo que $\lambda(t)$ asume el valor 55.

Solución (cont.)

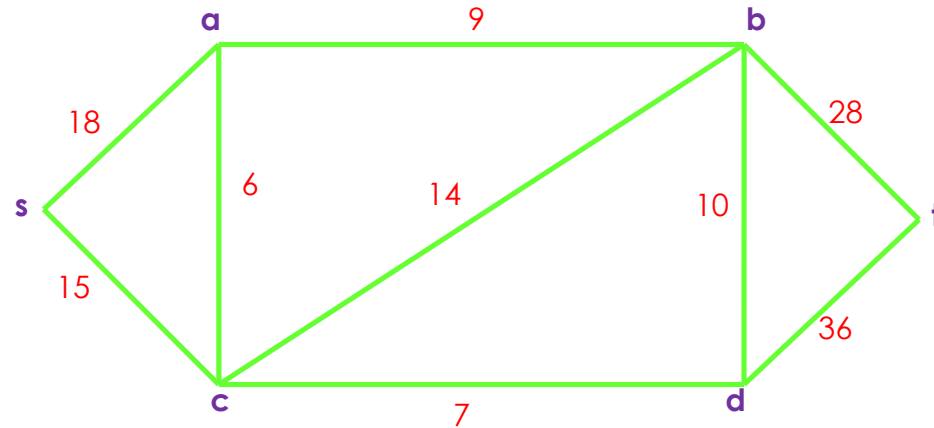


- Paso 5. $T = T - \{b\}$. Así, ahora tenemos

Vértice v	s	a	b	c	d	t
$\lambda(v)$	0	18	27	15	22	55
T	$\{t\}$					

Solución (cont.)

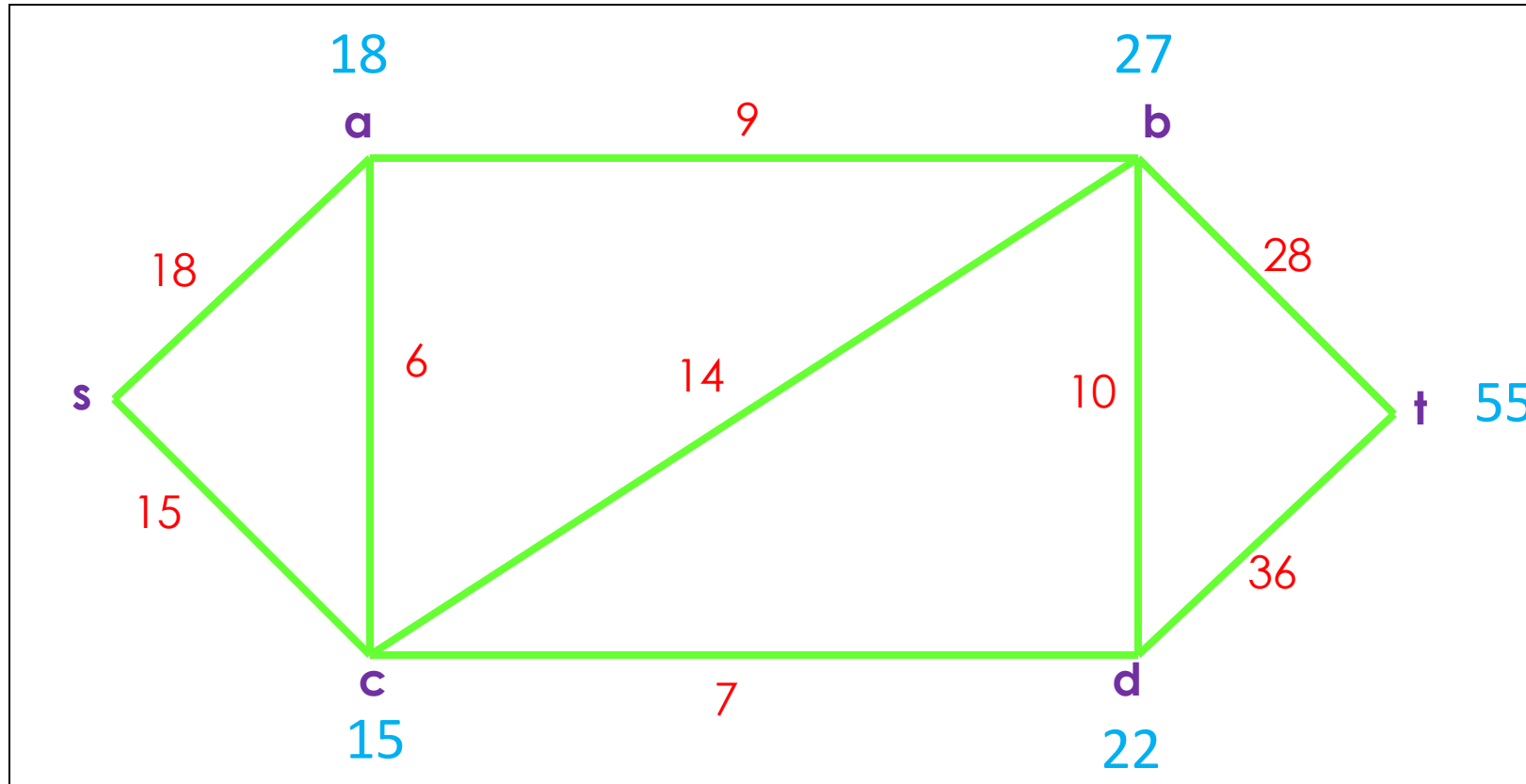
- Paso 2. $u = t$, la única elección posible.
- Paso 3. Parar.



Solución (cont.)

- Cuando el algoritmo termina, el valor $\lambda(v)$ representa la longitud del camino más corto desde el vértice s a cada vértice v .
- Las longitudes de esos caminos de s hasta a , b , c , d y t son 18, 27, 15, 22 y 55 respectivamente.

Ejemplo (valores finales calculados)



Solución

- Cuando el algoritmo termina, el valor $\lambda(v)$ representa la longitud de los caminos más cortos desde el vértice s a cada vértice v .

Búsqueda informada

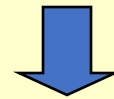
Búsqueda con Información

- Búsqueda Primero lo Mejor (BF)
- Búsqueda A*
 - Heurísticas

METODOS DE BÚSQUEDA CON INFORMACIÓN. BÚSQUEDA HEURÍSTICA

Al contar con información específica sobre un espacio de estados, se evita emprender búsquedas a ciegas

METODOS GENERALES + HEURISTICAS DE PROPOSITO ESPECIAL



MAYOR EFICIENCIA

ALGORITMO DE BÚSQUEDA GENERAL.

BÚSQUEDA GENERAL

responde con SOLUCIÓN o FALLA

LISTA-NODOS ← ESTADO INICIAL

Ciclo hacer

si LISTA-NODOS está vacía contestar FALLA

elegir NODO de LISTA-NODOS

si NODO es meta contestar con NODO

LISTA-NODOS ← expansión NODO

Fin Ciclo

FIN

MÉTODOS DE BÚSQUEDA CON INFORMACIÓN

Se utiliza una **FUNCIÓN HEURISTICA** para representar lo deseable que es la expansión de un nodo

f heurística: rep. de estados $\longrightarrow$ números

♣ Si *f* está bien diseñada, guía la búsqueda eficientemente.

♣ *f ideal* establece el camino a la meta.

MÉTODOS DE BÚSQUEDA CON INFORMACIÓN.

BÚSQUEDA PRIMERO EL MEJOR

(Best First Search)

*Los nodos se ordenan de tal manera que se **expande el nodo de mejor valor de la función heurística f** (mínimo, máximo), esta función puede incorporar conocimiento del dominio.*

♣ Algoritmo:

BUSQUEDA GENERAL donde LISTA-NODOS se ordena de acuerdo al valor de $f(\text{nodo})$

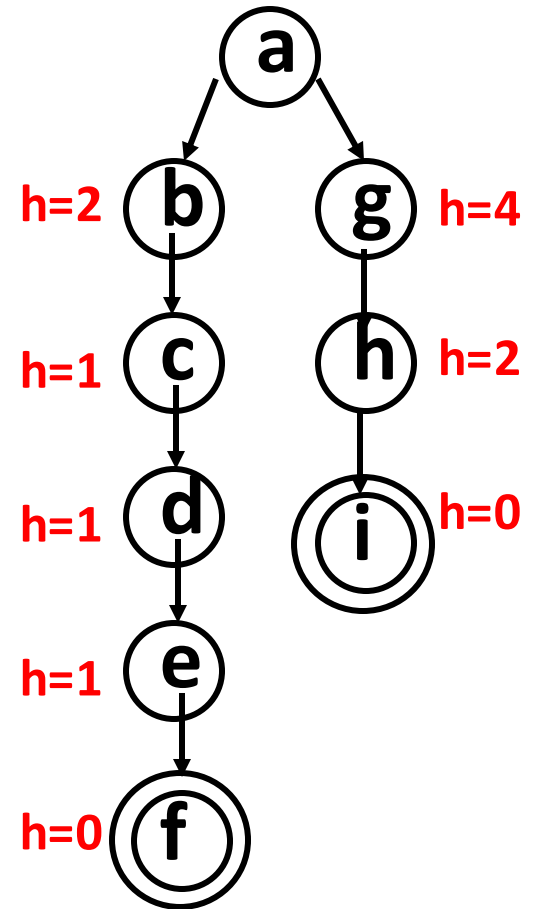
MÉTODOS DE BÚSQUEDA CON INFORMACIÓN.

BÚSQUEDA DE COSTO MÍNIMO (AVARA)

Implementa Búsqueda primero el mejor, buscando el mínimo de una función que representa el costo estimado para lograr una meta.

Búsqueda Avara - Ejemplo

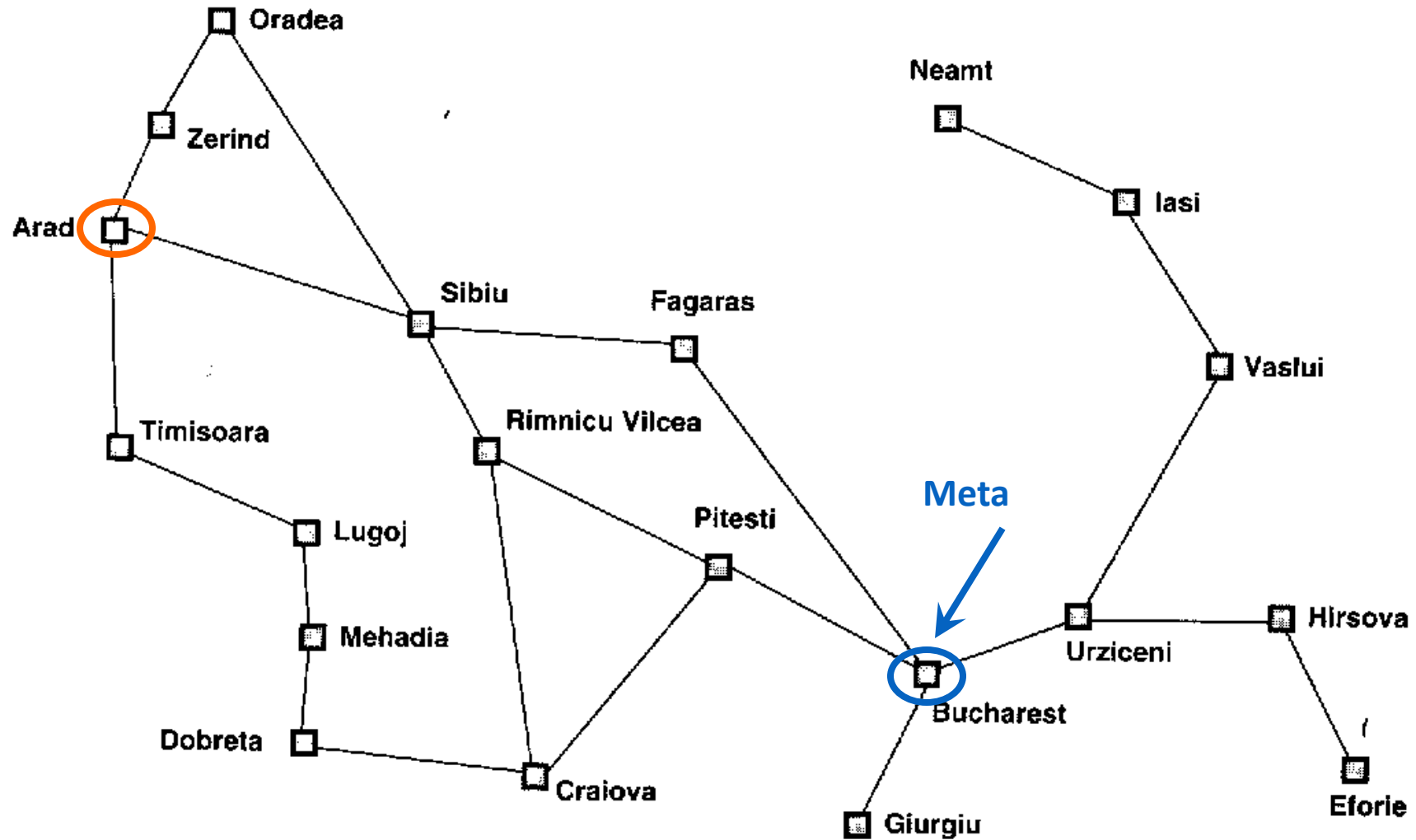
- Función de evaluación $f(n) = h(n)$
(la lista de los nodos se ordena de tal forma que **el nodo de mejor evaluación sea el primero**).
- Selecciona el nodo a expandir que se cree más cercano a un nodo meta (i.e., menor valor de $f = h$).
- No es óptima, como se ve en el ejemplo. Greedy search encontrará la meta f, que tiene un costo de 5, mientras que la solución óptima tiene un costo de 3.
- No es completa.



Búsqueda Avara

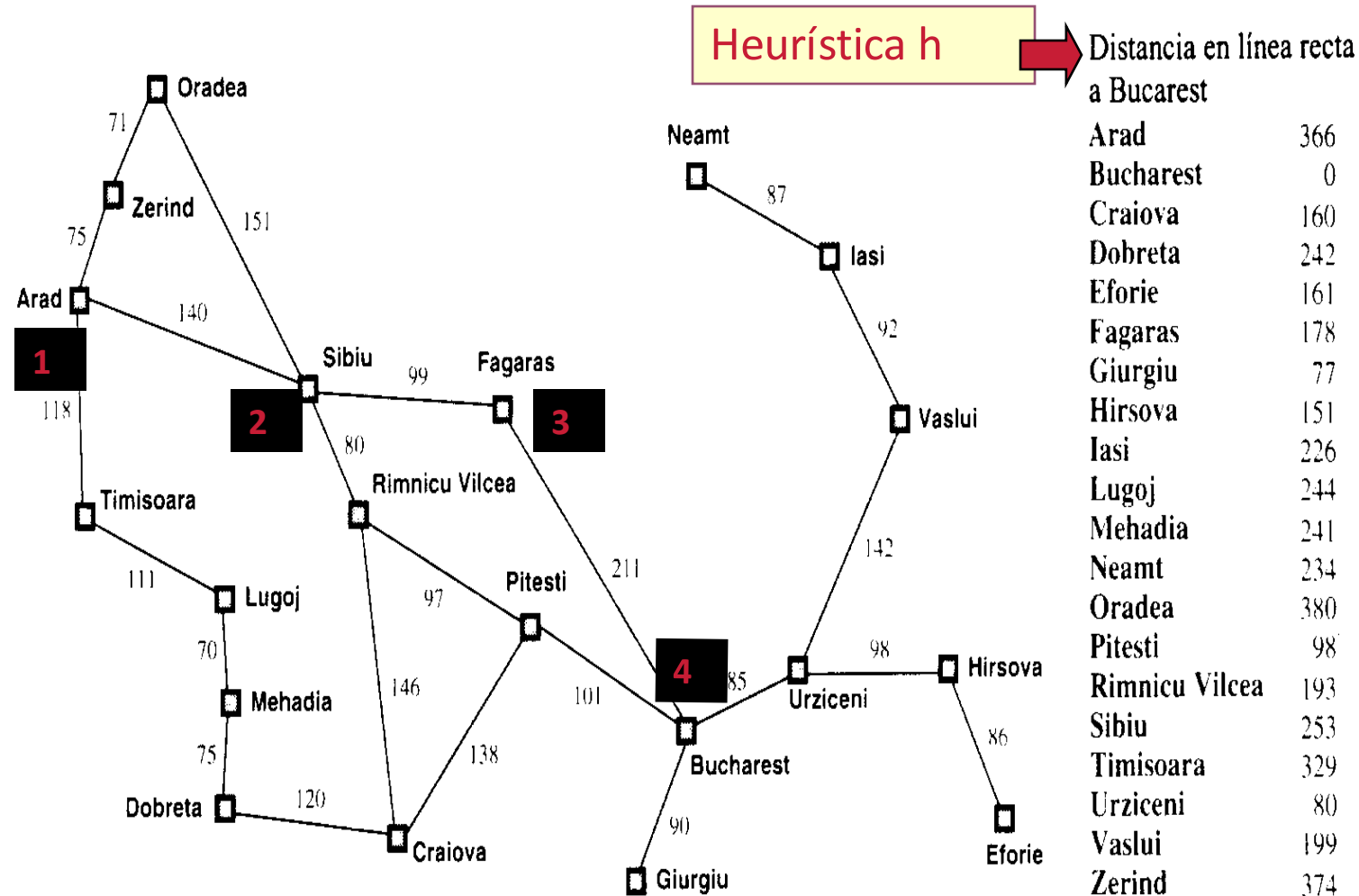
- Una de las búsquedas **Best First** más sencillas - mínimo costo estimado para llegar a la meta ($f = h$)
- Ese costo se puede estimar pero no determinar con exactitud; la buena heurística ayuda.
- $h(n)$ = costo estimado de la ruta más barata desde el estado n hasta el estado meta.
- Las funciones heurísticas son ***específicas para el problema***
- En problemas de búsqueda de ruta una buena h es $h(DLR)$, donde DLR es distancia en línea recta

Ejemplo: Encontrar Camino de Arad a Bucarest

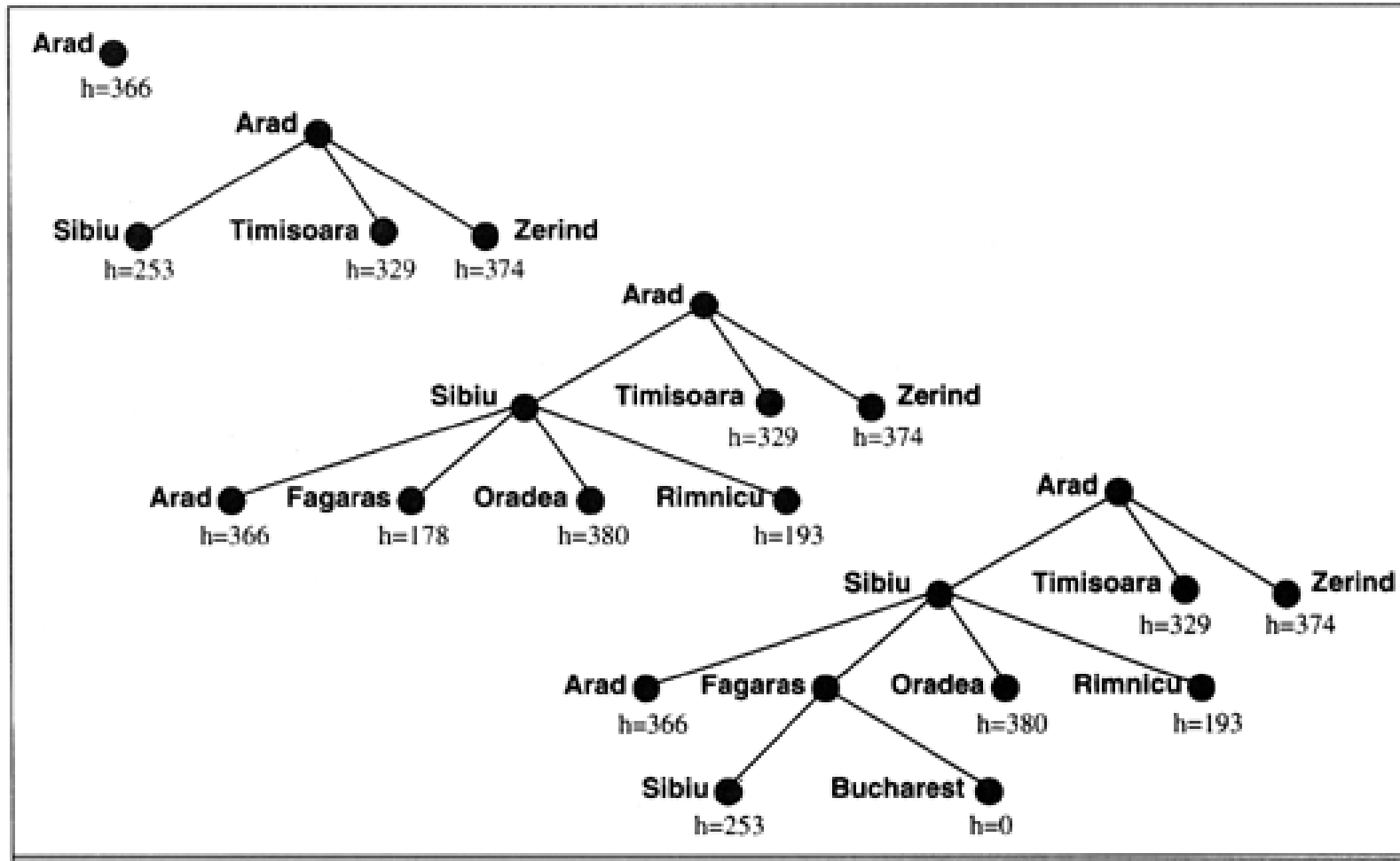


BUSQUEDA AVARA

Ejemplo: Encontrar Camino de Arad a Bucarest



ÁRBOL DE BÚSQUEDA PARCIAL (Arad a Bucarest).



Búsqueda Avara

- En este ejemplo, la búsqueda avara produce un costo de búsqueda mínimo - no expande nodos fuera de la ruta solución
- No es la ruta óptima (ruta por Rimmicu-Pitesti es más corta)
- Desempeño bastante bueno, tienden a encontrar soluciones rápidamente

Ejemplos de problemas reales

- Problemas del mundo real
 - determinación de una ruta
 - problema del vendedor viajero
 - distribución VLSI (very large scale integration)
 - navegación de un robot
 - secuencia de ensamblaje

Vendedor viajero

- estados
 - ubicaciones / ciudades
 - estados ilegales
 - cada ciudad debe ser visitada sólo una vez
- operadores
 - mover desde una ubicación a otra
- test de objetivo
 - todas las ubicaciones visitadas
 - agente en la ubicación inicial
- costo del camino
 - distancia entre ubicaciones

Búsqueda en Profundidad Limitada

- Similar a búsqueda en profundidad, pero con un límite
 - basado en el método BUSQUEDA-GENERAL
 - requiere un contador para indicar la profundidad

Búsqueda Inversa (Backward Chaining)

- Es una forma de cambiar la perspectiva del problema para realizar la búsqueda
- Consiste en implementar la búsqueda haciendo que el estado objetivo juegue el papel de estado inicial y vice versa
- Aquí se usa cualquiera de los métodos anteriores

Búsqueda BF (Best First Search)

- Función de evaluación de los nodos
 - Permite decidir acerca del **mejor nodo** a ser expandido
 - hay una familia de métodos de búsqueda con varias funciones de evaluación
 - generalmente da una estimación de la **distancia** al **objetivo**
 - a menudo es llamada *heurística* en este contexto
 - El nodo con el valor más bajo se expande primero

Búsqueda BF (Best-First Search)

- Función de evaluación de los nodos
 - Permite decidir acerca del **mejor nodo** a ser expandido
 - hay una familia de métodos de búsqueda con varias funciones de evaluación
 - generalmente da una estimación de la **distancia** al **objetivo**
 - a menudo es llamada *heurística* en este contexto
 - el nodo con el valor más bajo se expande primero

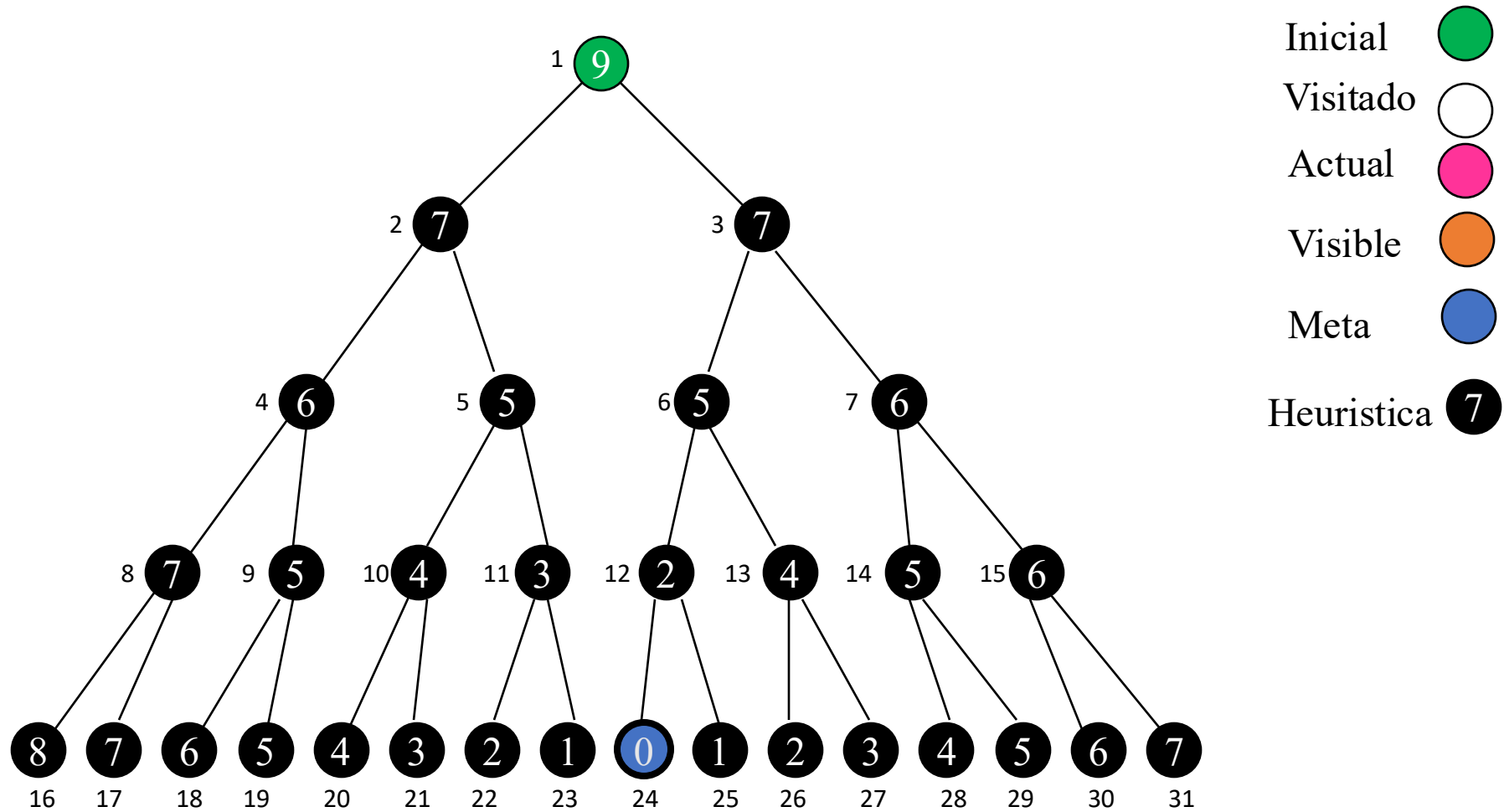
Búsqueda Greedy

- Minimiza el costo estimado a un objetivo
 - **expande** el nodo que parece más cercano a la meta
 - utiliza una función **heurística**
 - $h(n)$ = costo estimado desde el nodo actual hasta la meta
 - las **funciones heurísticas** son propias de un problema específico
 - a menudo la distancia en línea recta para encontrar rutas y problemas similares

Búsqueda Greedy

- Ventaja
 - A menudo mejor que búsqueda en profundidad,
- Desventaja
 - Complejidad y espacio generalmente son iguales o peores

Instantánea Búsqueda Greedy

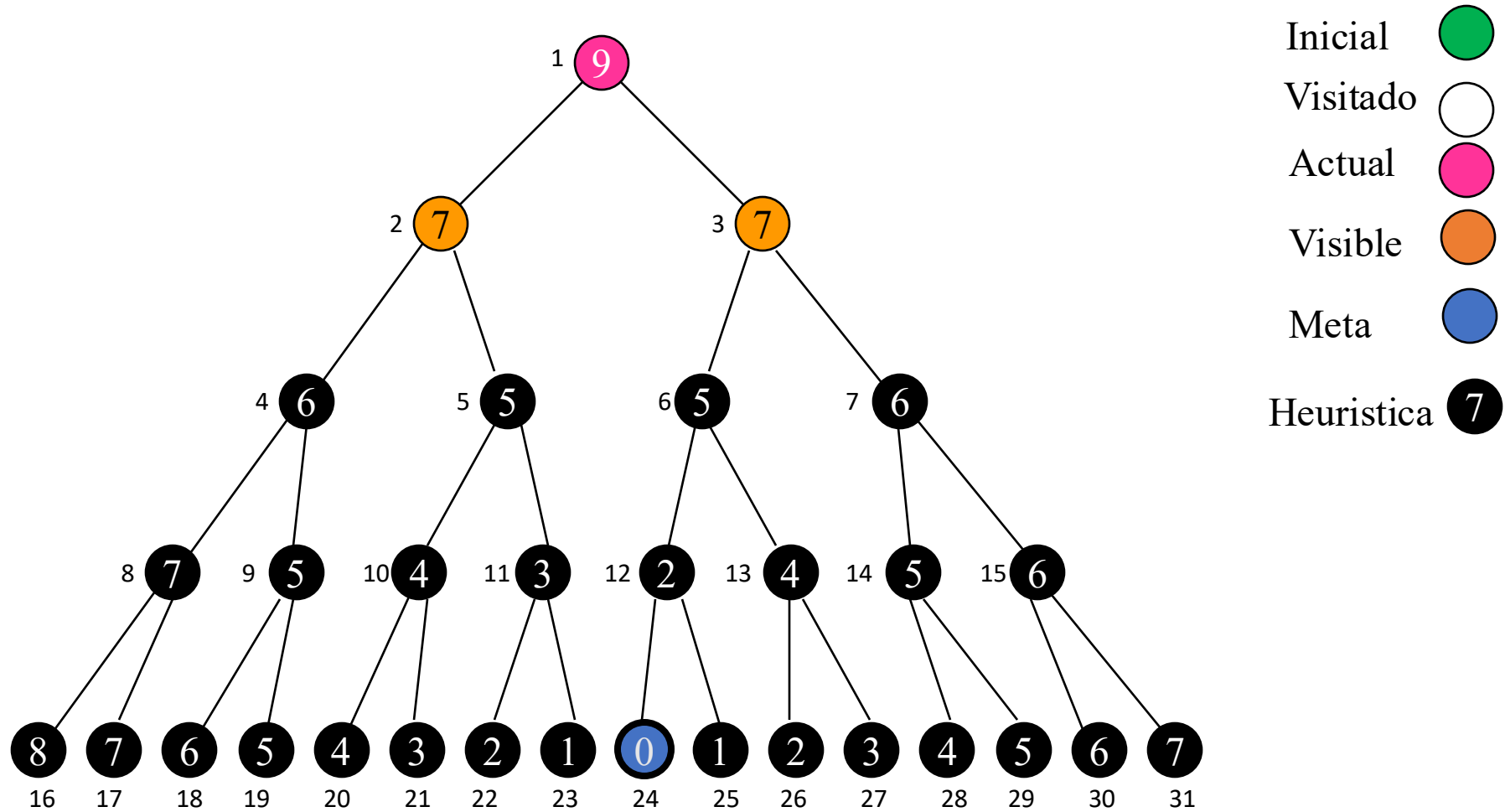


nodos: [1(9)]

cerrados: []

Actual=

Instantánea Búsqueda Greedy

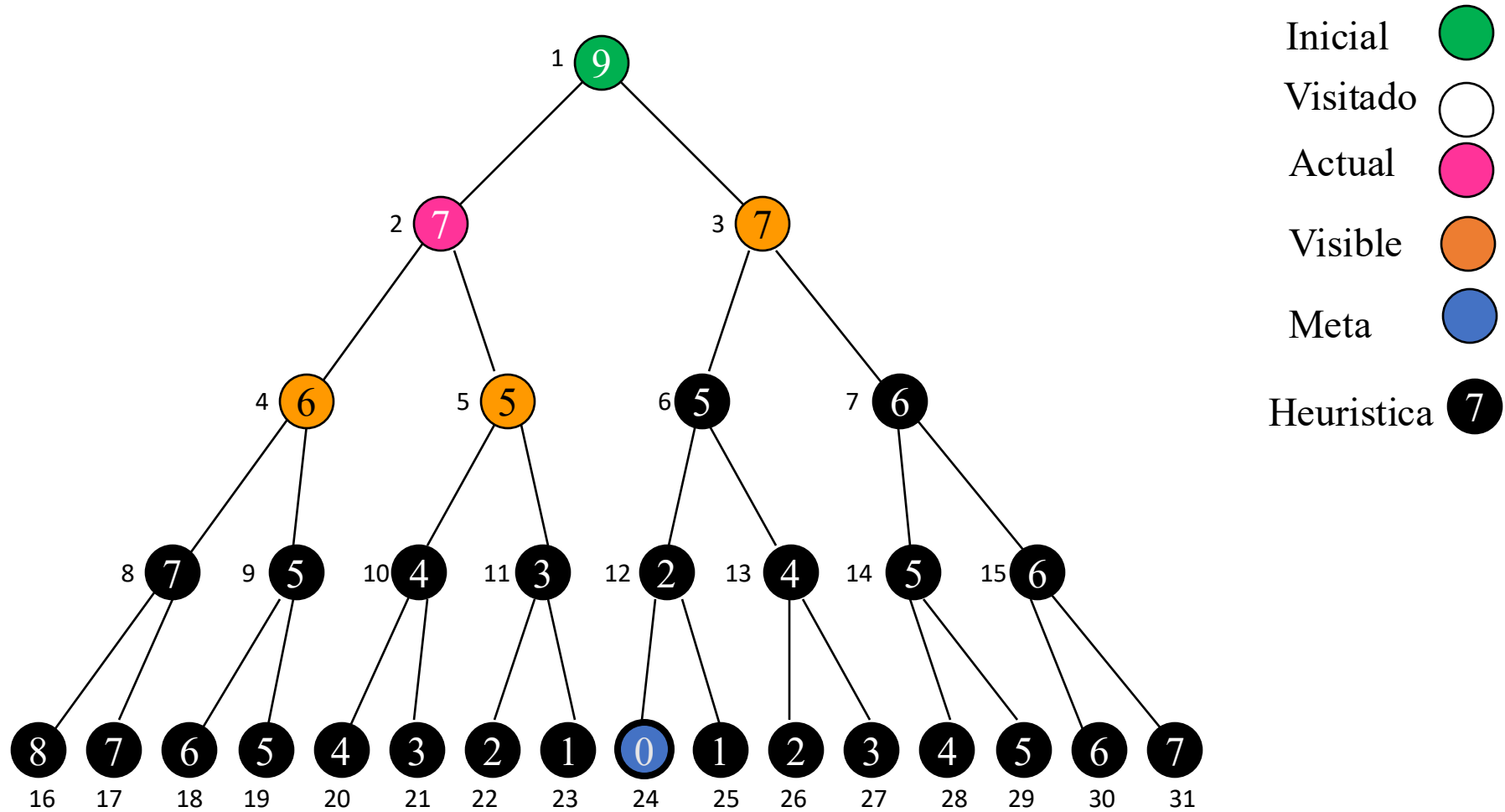


nodos: [2(7),3(7)]

cerrados: []

Actual=1(9)

Instantánea Búsqueda Greedy

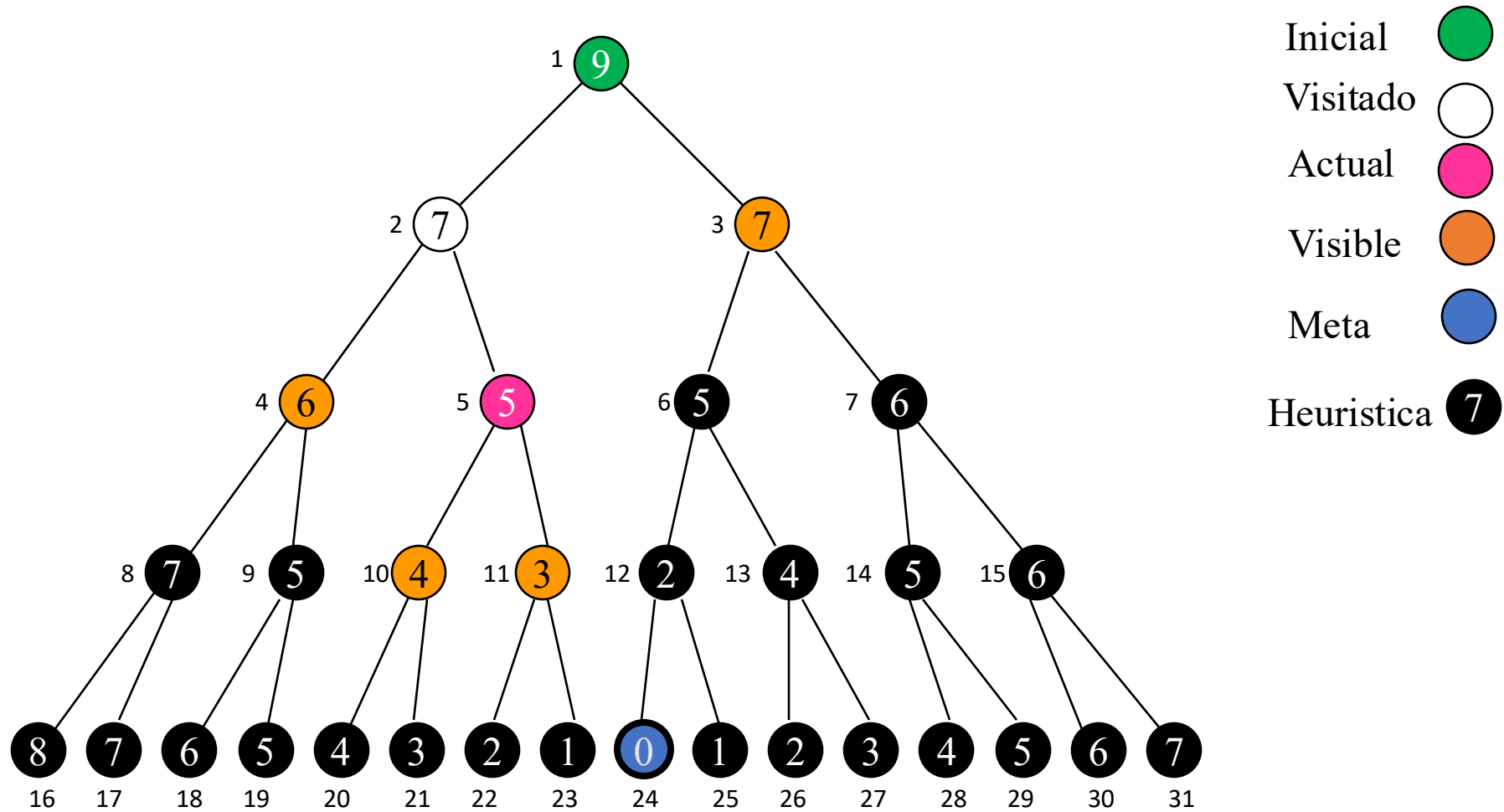


nodos: [3(7),4(6),5(5)]

cerrados: [1(9)]

Actual=2(7)

Instantánea Búsqueda Greedy

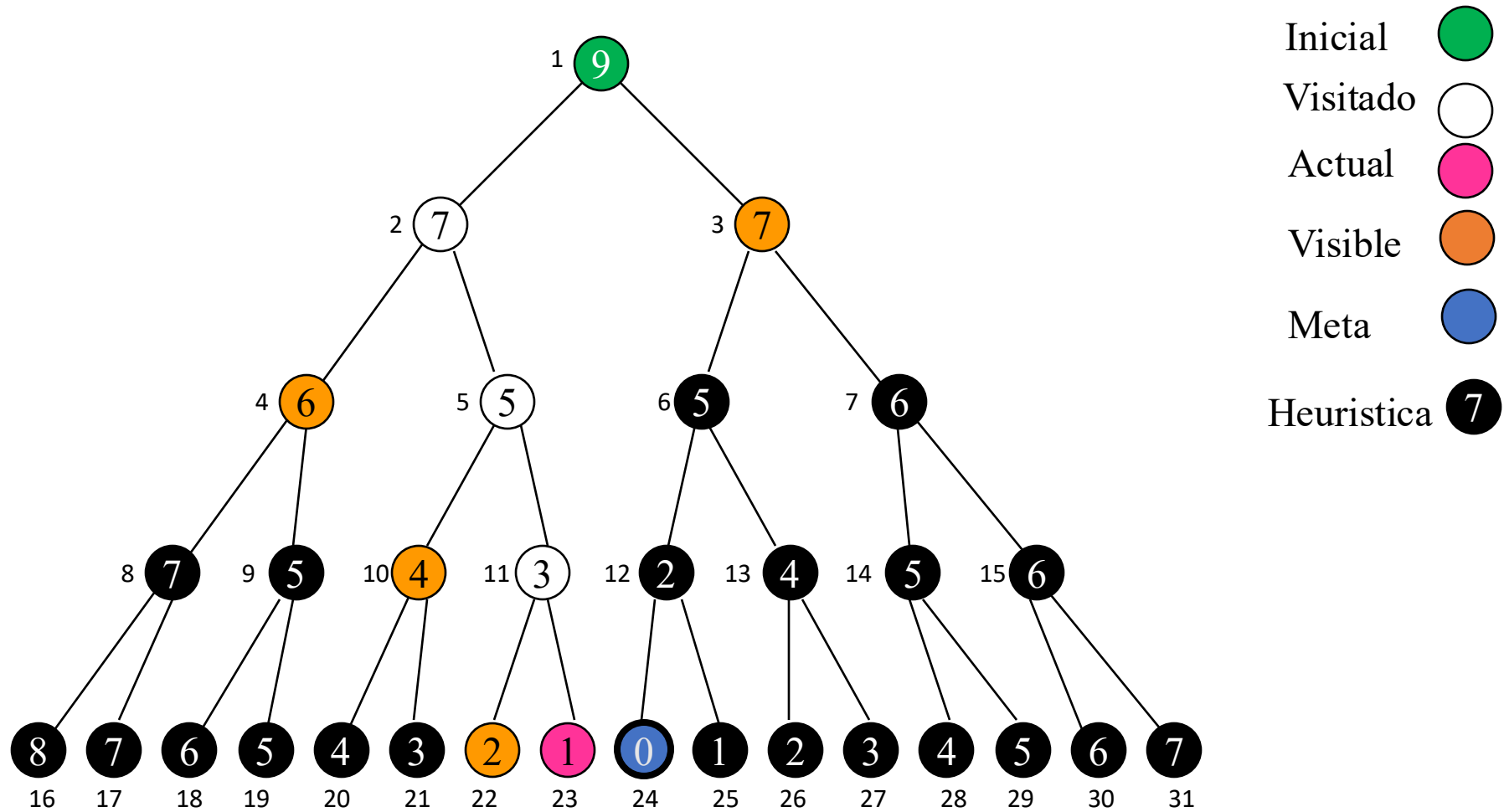


nodos: [3(7),4(6),10(4),11(3)]

cerrados: [1(9),2(7)]

Actual=5(5)

Instantánea Búsqueda Greedy

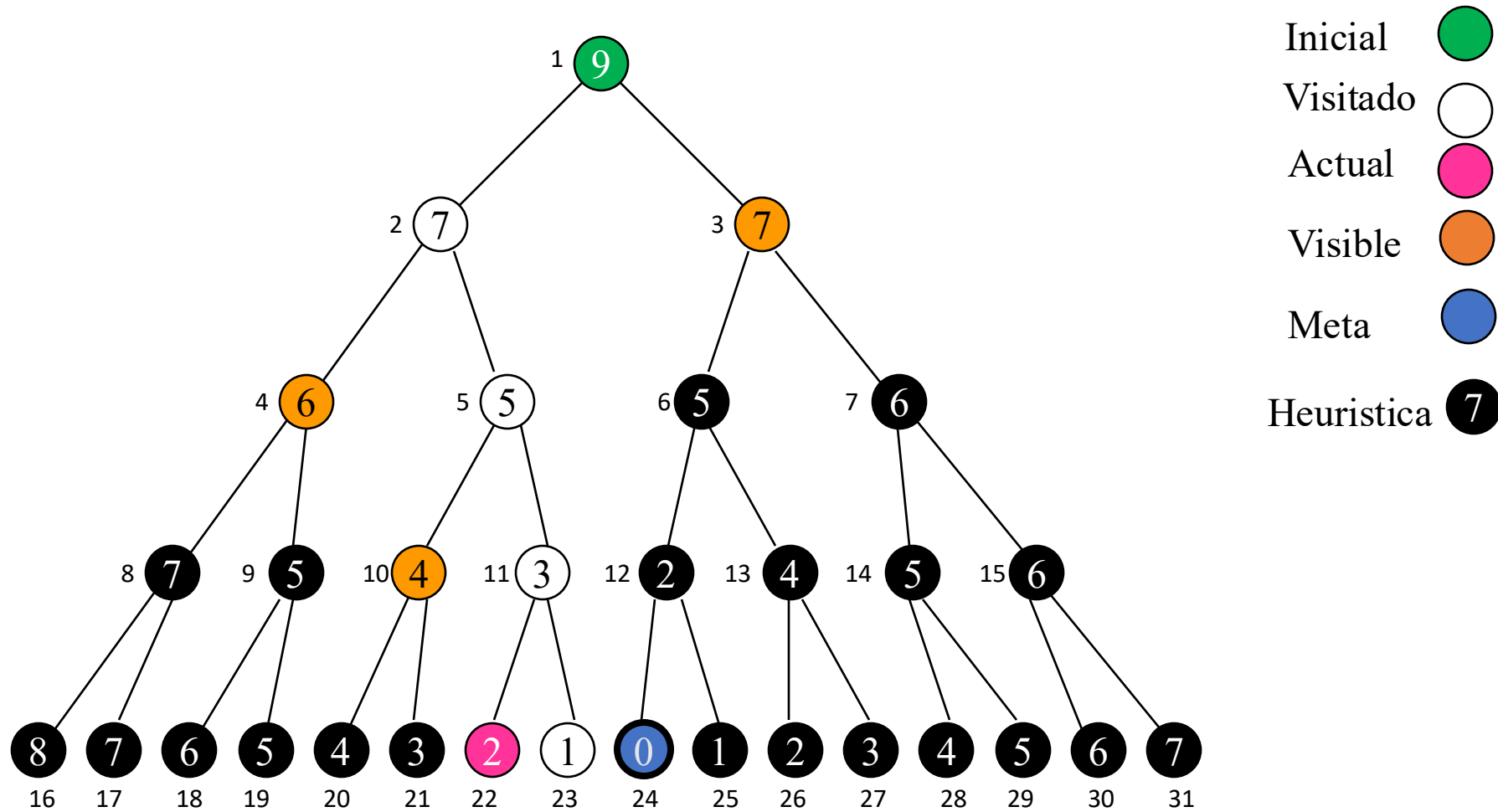


nodos: [3(7),4(6),10(4),22(2)]

cerrados: [1(9),2(7),5(5),
11(3)]

Actual= 23(1)

Instantánea Búsqueda Greedy

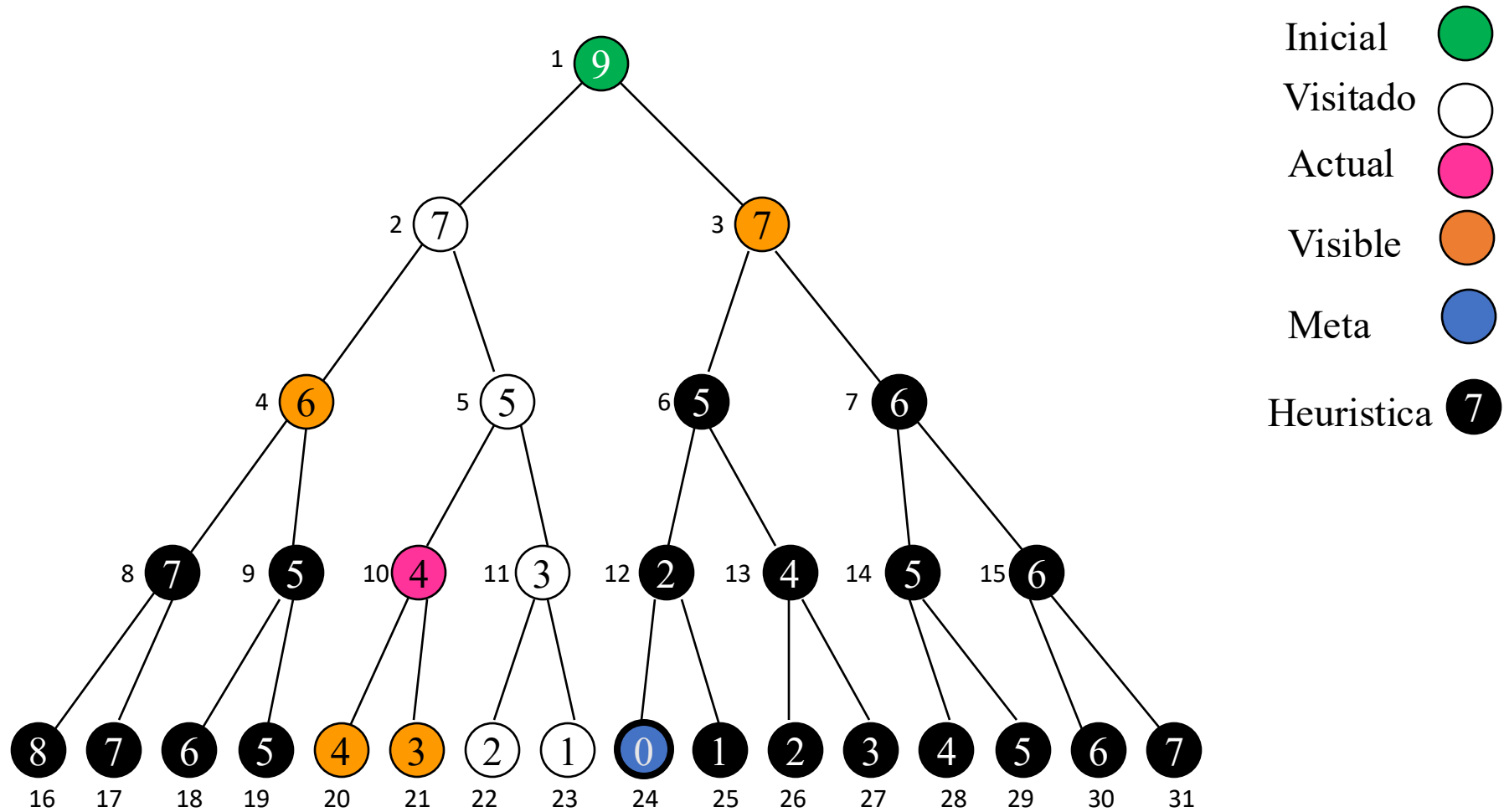


nodos: [3(7),4(6),10(4)]

cerrados: [1(9),2(7),5(5),
11(3), 23(1)]

Actual= 22(2)

Instantánea Búsqueda Greedy

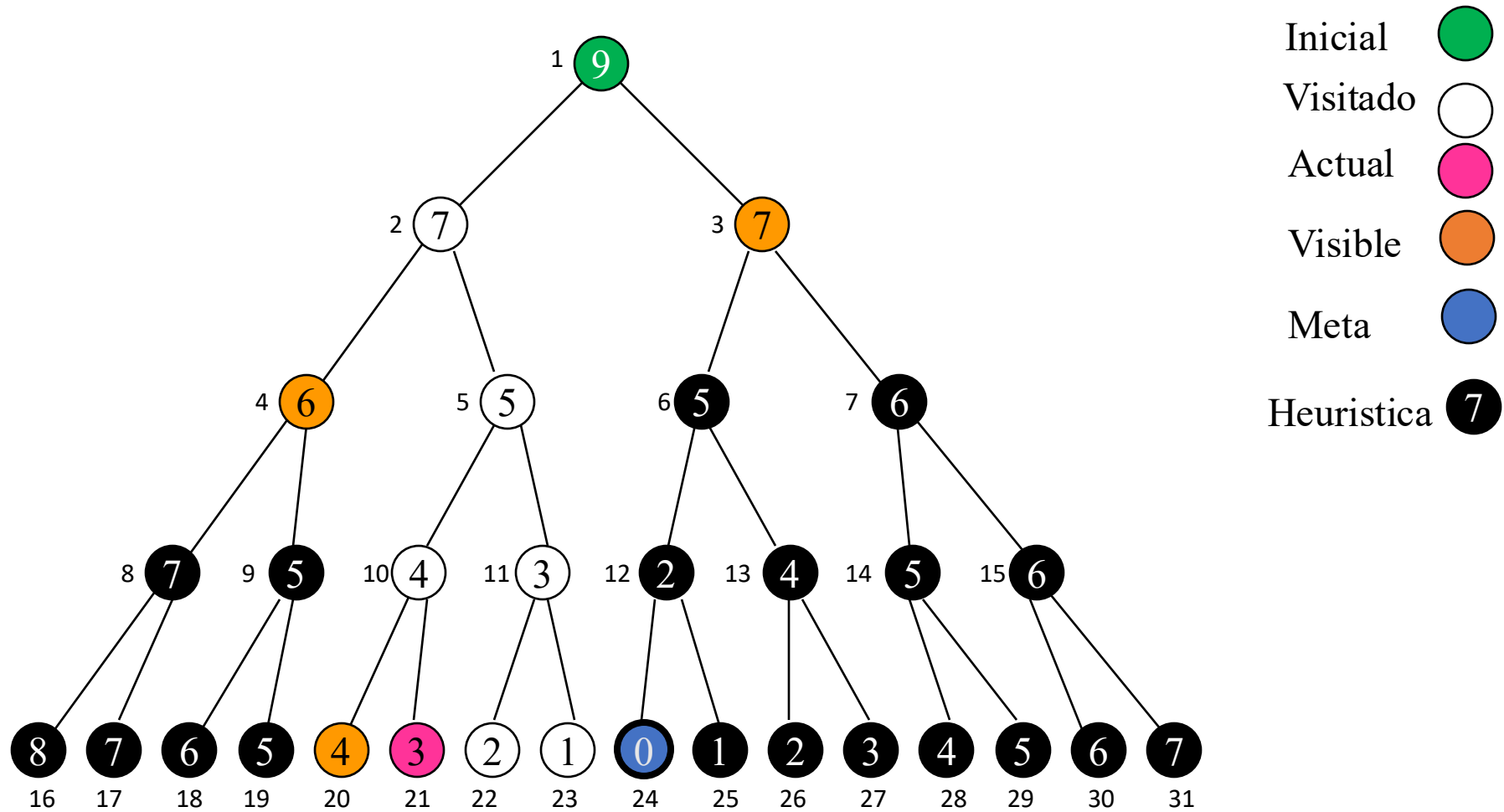


nodos: [3(7),4(6),20(4),21(3)]

cerrados: [1(9),2(7),5(5),
11(3),23(1),22(2)]

Actual= 10(4)

Instantánea Búsqueda Greedy

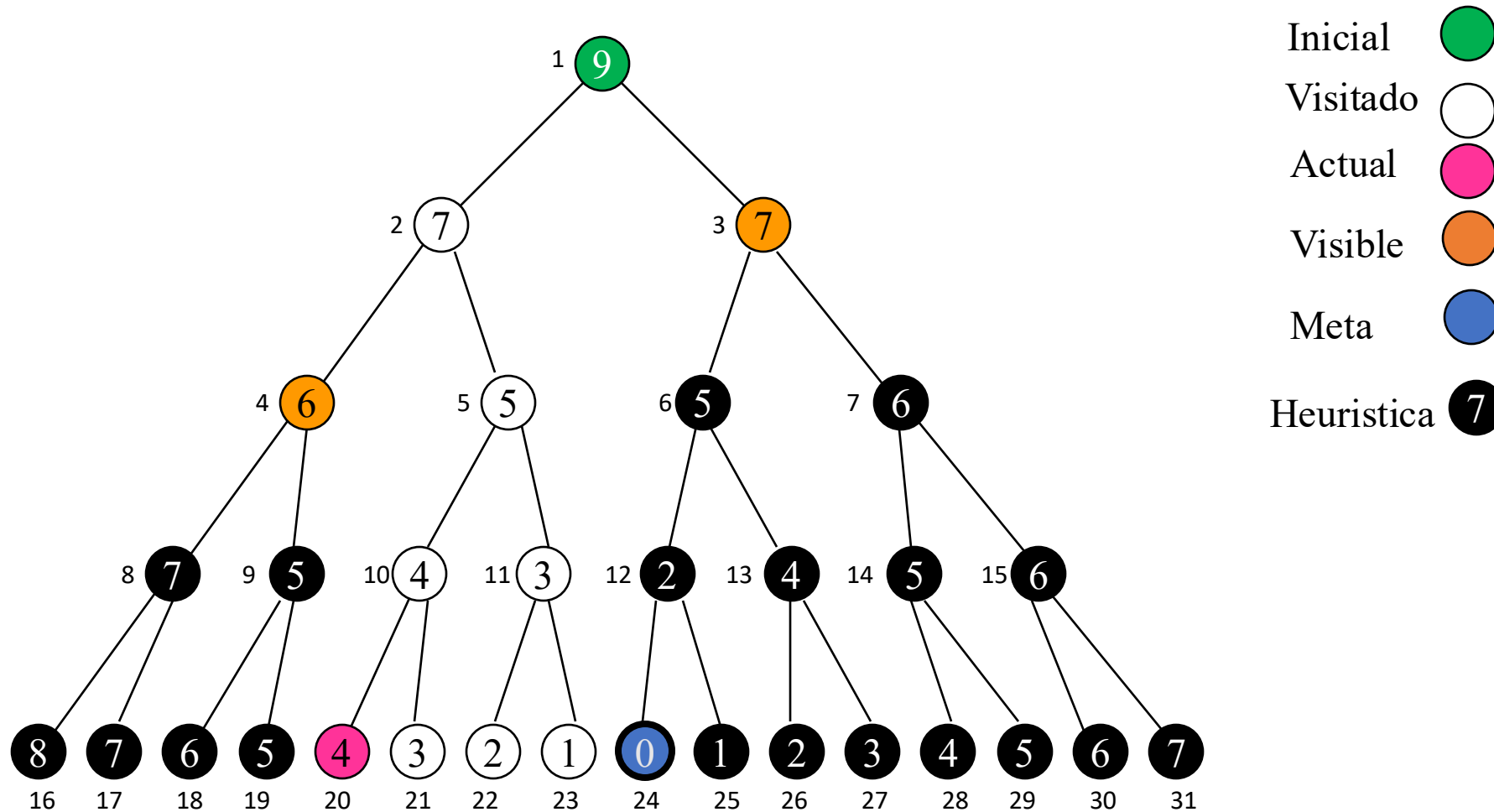


nodos: [3(7),4(6),20(4)]

cerrados: [1(9),2(7),5(5),
11(3),23(1),22(2),10(4)]

Actual= 21(3)]

Instantánea Búsqueda Greedy

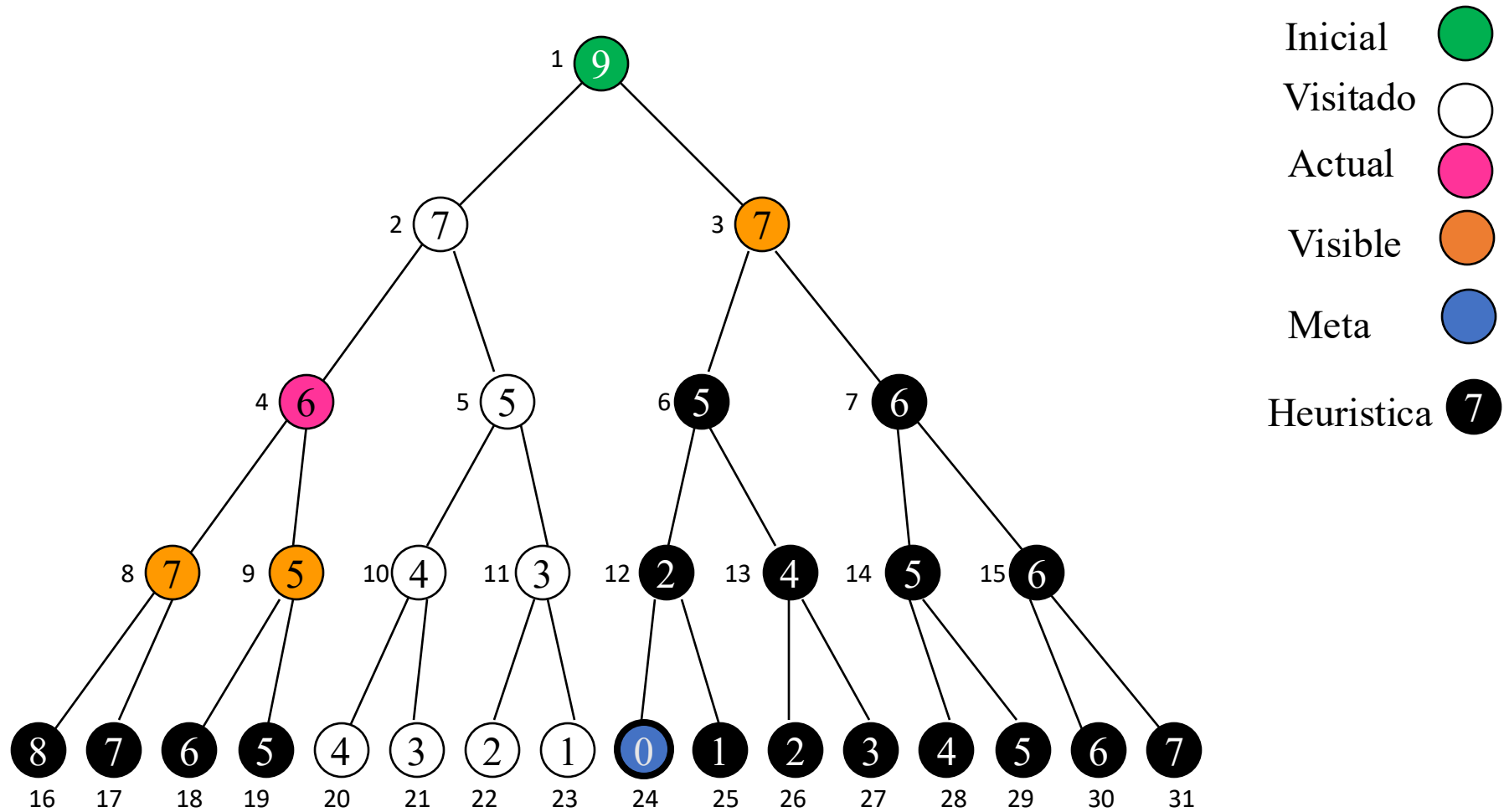


nodos: [3(7),4(6)]

cerrados:
[1(9),2(7),5(5),11(3),23(1),
22(2),10(4), 21(3)]

Actual= 20(4)

Instantánea Búsqueda Greedy

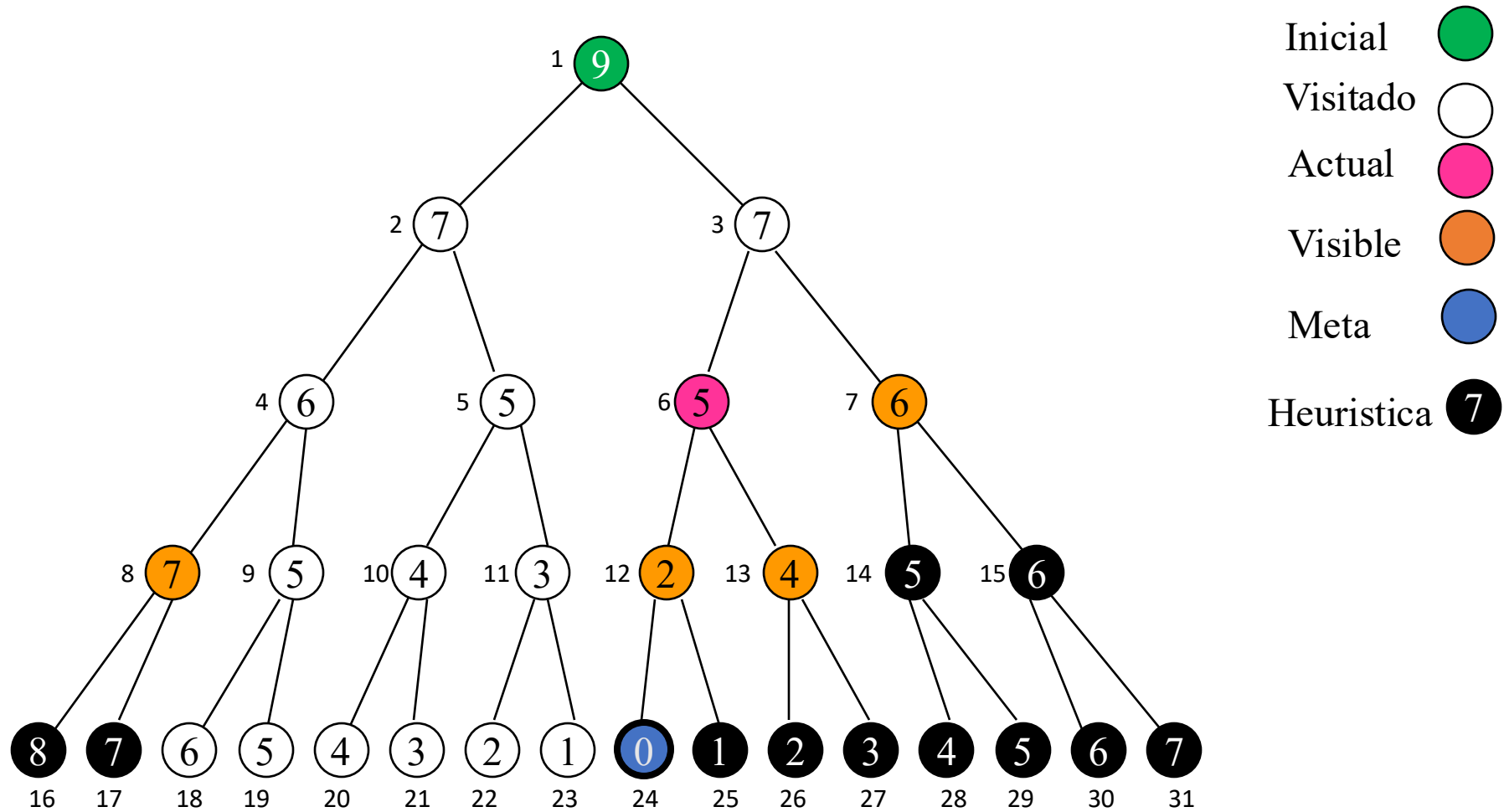


nodos: [3(7),8(7),9(5)]

cerrados:
[1(9),2(7),5(5),11(3),23(1),
22(2),10(4),21(3),20(4)]

Actual= 4(6)

Instantánea Búsqueda Greedy



nodos: [8(7),7(6),12(2),13(4)]

cerrados: [1(9),...,4(6),9(5),
19(5),18(6),3(7)]

Actual=6(5)

Búsqueda A*

- combina búsqueda greedy y costo estimado para encontrar la trayectoria más barata a través del nodo actual
 - $f(n) = g(n) + h(n)$
= costo camino + costo estimado a la meta
 - heurística debe ser admisible
 - nunca sobre-estimar el costo de alcanzar la meta
 - muy buen método de búsqueda, pero con problemas de complejidad

Propiedades de A^*

- El valor de f nunca disminuye a lo largo del camino desde el nodo inicial
 - también se conoce como monotonidad de la función
 - casi todas las heurísticas admisibles muestran monotonidad
 - las que no, pueden ser modificadas sin mucho esfuerzo
- esta propiedad puede ser usada para generar contornos
 - regiones donde el f -costo está bajo un cierto umbral
 - con costo uniforme ($h = 0$), los contornos son circulares
 - a mejor heurística h , más preciso es el contorno en torno al camino óptimo

Optimalidad de A^*

- A^* encontrará la solución optimal
 - la primera solución encontrada es la optimal
- A^* es optimalmente eficiente
 - no hay otro algoritmo que garantice la expansión de menos nodos que A^*

Complejidad de A^*

- El número de nodos dentro del espacio de búsqueda todavía es exponencial
 - con respecto a la longitud de la solución
 - mejor que otros algoritmos, pero todavía problemático
- Con frecuencia, la complejidad espacial es más severa que la complejidad temporal
 - A^* guarda todos los nodos generados en memoria

Búsqueda con Memoria Acotada

- Algoritmos de búsqueda que tratan de ahorrar memoria
- La mayoría son variaciones de A^*
 - profundización iterativa de A^* (IDA*)
 - memoria acotada simplificada A^* (SMA*)

Heurísticas de búsqueda

- Para muchas tareas, una buena heurística es la clave para encontrar una solución
 - poda del espacio de búsqueda
 - se mueve hacia la meta
- Relajación de problemas
 - menos restricciones a los operadores
 - su solución exacta puede ser una buena heurística para el problema original

Heurística Puzzle-8

- **Nivel de dificultad**
 - alrededor de 20 pasos para una solución típica
 - factor de ramificación aproximadamente 3
 - búsqueda exhaustiva lleva a $3^{20} \approx 3.5 * 10^9$ movimientos
 - $9! = 362,880$ diferentes configuraciones para las 9 piezas
- **Candidatas a función heurística**
 - número de piezas fuera de lugar
 - suma de las distancias de las piezas a sus posiciones correctas
- **Generación de heurísticas**
 - posible a partir de especificaciones formales

Heurísticas de búsqueda

- Para muchas tareas, una buena heurística es la clave para encontrar una solución
 - poda del espacio de búsqueda
 - se mueve hacia la meta
- Relajación de problemas
 - menos restricciones a los operadores
 - su solución exacta puede ser una buena heurística para el problema original

BÚSQUEDA A*

Condiciones para que A* sea completa y optimal:



➤ ***h sea aceptable:***

h no sobreestime el costo a la meta

f no sobrestima el costo real de la solución

Ejemplo: h = distancia en línea recta

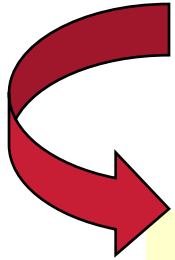
➤ ***f es monótona:***

si nunca disminuye a través de una ruta que parte de la raíz: $f(\text{padre}(n)) \leq f(n)$

Optimalidad de A^*

- Definir f^* - el costo de la solución óptima para la ruta
 - A^* expande todos los nodos con $f(n) < f^*$
 - A^* podría expandir algunos de los nodos para los cuales $f(n) = f^*$, antes de seleccionar el estado meta.
- La primera solución encontrada debe ser la óptima, dado que los nodos de todos los contornos subsiguientes tendrán un costo más alto (todos los estados meta tienen $h(n) = 0$).

VENTAJAS Y DESVENTAJAS BÚSQUEDA A*



- El uso de una heurística buena provee ventajas enormes
- Usualmente A* se queda sin espacio antes de quedarse sin tiempo, puesto que mantiene a todos los nodos en memoria
- ❖ *Problema de memoria > problema del tiempo*
- ❖ *Problema de encontrar buenas heurísticas h*

Conclusiones - Búsqueda

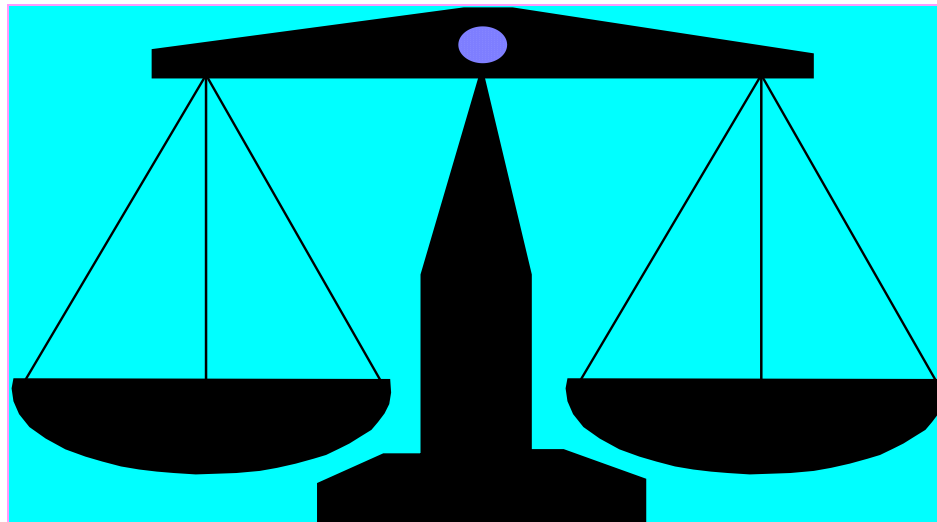
- Si no sabemos cómo obtener X , creemos un espacio de estados donde sepamos que va a estar incorporado X y luego busquemos a X dentro de ese espacio.
- Mérito de esta formulación $\Rightarrow$ siempre es posible encontrar un espacio donde esté contenida la respuesta o solución.
- Cuanto menos conocimiento tengamos, tanto más grande será el espacio.
 - **La incorporación de heurísticas lo reduce!!!**

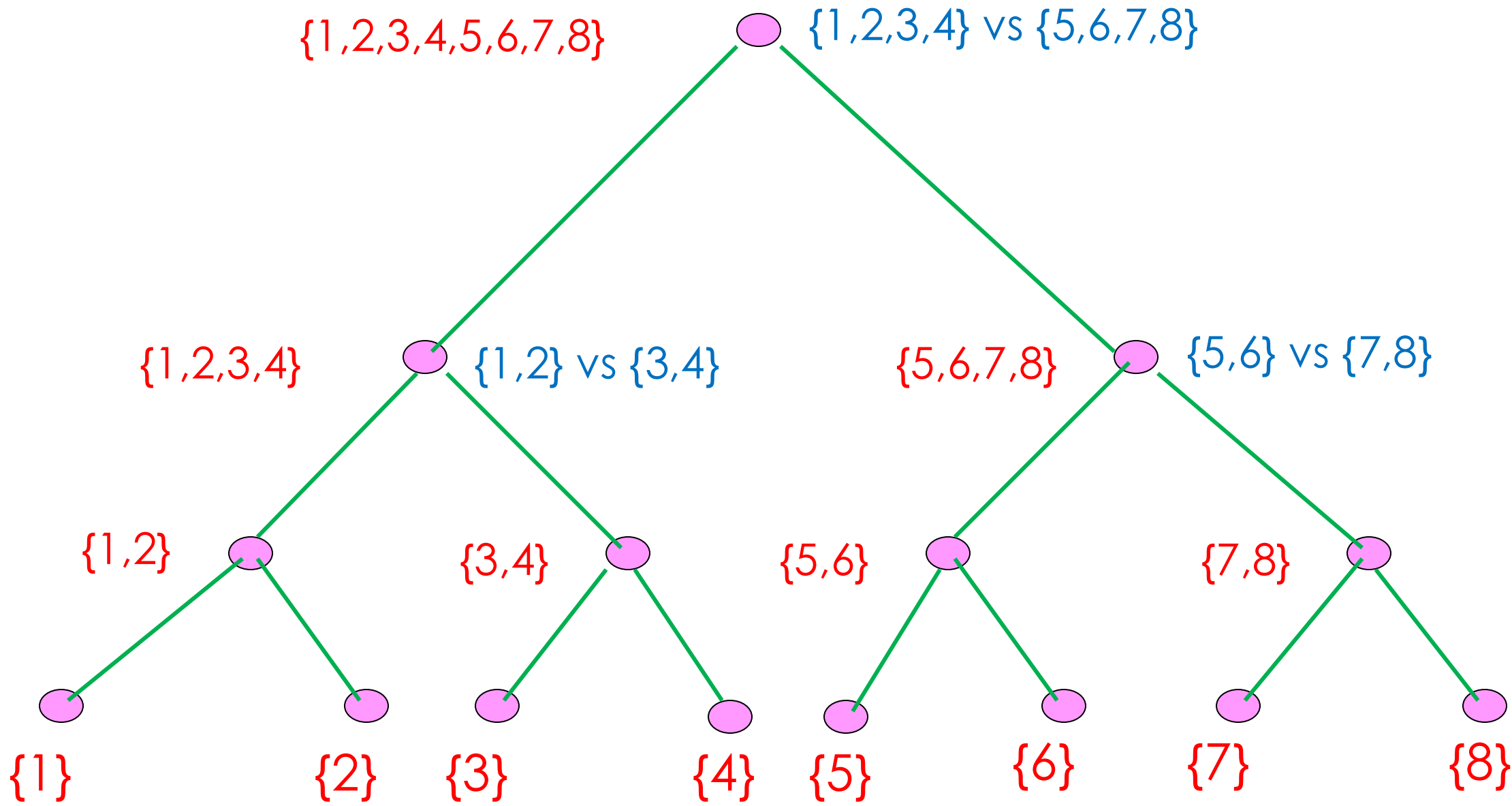
Arboles de Decisión

- Un árbol de decisión es un grafo en el que cada nodo está asociado a un estado y cada arista está asociada a una entrada o acción que provoca la transición.
- Son una forma conveniente de enumeración del conjunto de posibles caminos en un procedimiento solución.

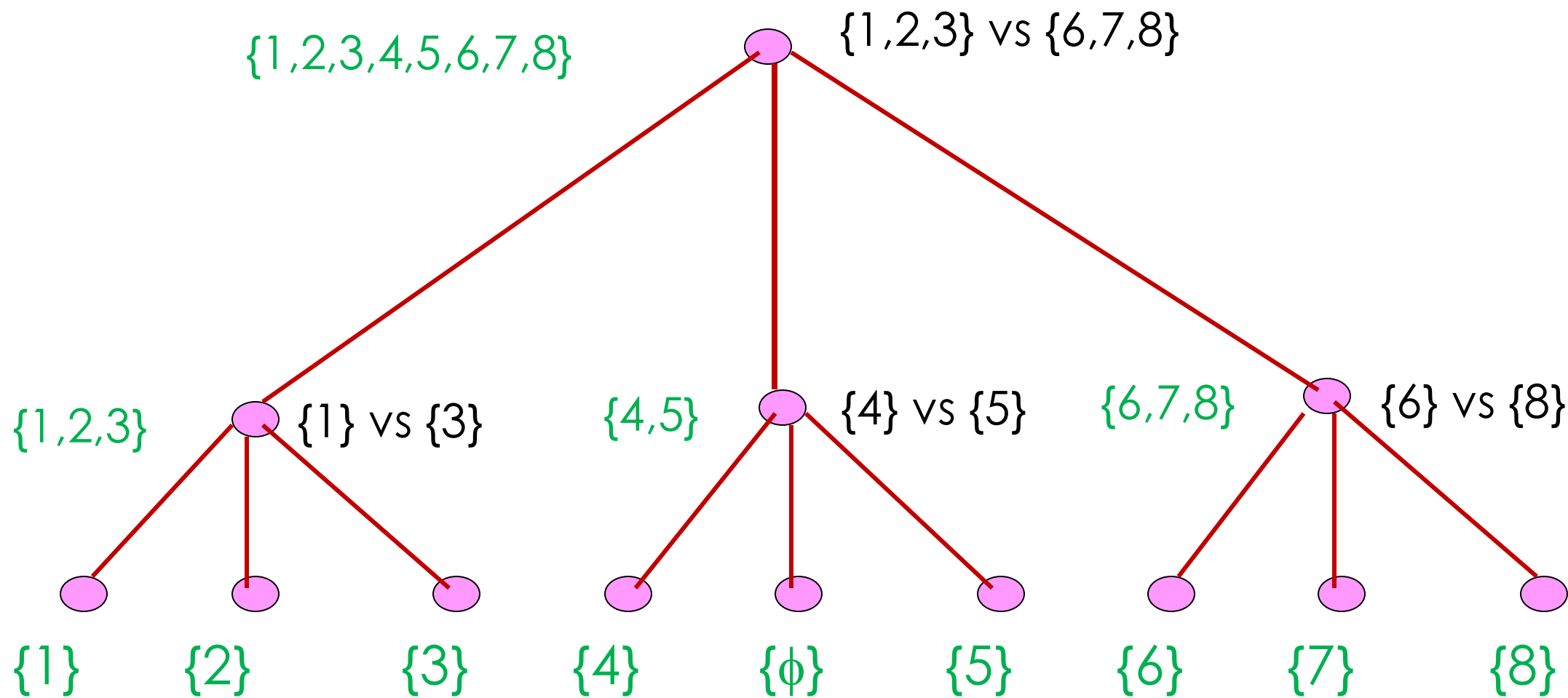
- Cada nodo interno de un árbol de decisión corresponde a una solución parcial; cada hoja corresponde a una solución.
- La ejecución de un procedimiento solución corresponde a recorrer una trayectoria desde la raíz del árbol hasta una hoja. La longitud de esa trayectoria indica el número de tests necesarios para ejecutar un procedimiento.

- Ejemplo. Supongamos que tenemos 8 monedas, de las cuales se sabe que hay una falsa (y más pesada que las otras). Se nos pide encontrar la moneda falsa usando solamente una balanza para comparar los pesos de dos conjuntos de monedas.





- El árbol anterior requiere tres “pesas” para encontrar la moneda falsa. ¿Es posible hacer una búsqueda más eficiente?
- Nos interesa un procedimiento en el cual el máximo número de pasos del algoritmo sea tan pequeño como sea posible.
- Un procedimiento **minimax**, es uno que minimiza el máximo número de pasos requeridos para resolver un problema.



- La altura del árbol de decisión de un procedimiento minimax no es mayor que la altura de un árbol de decisión para cualquier algoritmo que resuelva el problema.
- Las técnicas básicas de conteo se pueden usar para encontrar los límites al número de pasos en la solución minimax de una tarea.

Comportamiento asintótico de funciones

- Una de las maneras de evaluar un programa es escoger un conjunto de entrada “típico” y encontrar cuán rápido resuelve un problema.
- Estas medidas empíricas están influenciadas tanto por el programa como por la máquina usada para implementar el algoritmo.
 - Si dos programas son comparados en computadores diferentes, los resultados también pueden ser diferentes.

Un ejemplo para entender el concepto

- Supongamos que un cierto algoritmo A ordena una secuencia de n números en forma ascendente, intercambiando, de alguna manera, pares de números; y supongamos además que puede demostrarse que el número exacto de pasos que el algoritmo A ejecuta es $t(n) = 3 + 6 + 9 + \dots + 3n$
- ¿Cuál es el orden del algoritmo A?

- Es posible decir que el algoritmo A ejecuta en $O(n^2)$ pasos, según el siguiente razonamiento:
- $t(n)$ puede escribirse como un término general ya que

$$t(n) = \sum_{i=1}^n 3i = 3n(n+1)/2$$

- Tenemos que demostrar que existen $f(n)$, C y n_0 tales que $t(n) \leq C \cdot f(n)$ para todo $n \geq n_0$

- Si elegimos $C = 3$, $n_0 = 1$ y $f(n) = n^2$
- Tenemos que demostrar que se cumple $3 \cdot n \cdot (n+1)/2 \leq 3 \cdot n^2$
- En efecto, multiplicando por $2/3$ se tiene que $n^2 + n \leq 2 \cdot n^2$, o sea $n \leq n^2$, lo que es válido para todo $n > 1$

Un cálculo

- Supongamos que vamos a ordenar 10^6 números. Usaremos un algoritmo $O(n^2)$ para un supercomputador y un algoritmo $O(n \log n)$ para un PC. Digamos que los $t(n)$ respectivos son $2n^2$ y $50 n \log n$.
- El supercomputador ejecuta 100.000.000 instrucciones por segundo. El PC ejecuta 1.000.000 de instrucciones por segundo

- El supercomputador demorará:

$$\frac{2 \cdot (10^6)^2 \text{ instrucciones}}{10^8 \text{ instrucciones/segundo}}$$

$$= 20.000 \text{ segundos}$$

$$\approx 5,56 \text{ horas}$$

- El PC demorará:

$$\frac{50 \cdot 10^6 \log_2 10^6 \text{ instrucciones}}{10^6 \text{ instrucciones/segundo}}$$

$$= 996,677 \text{ segundos}$$

$$\approx 16,61 \text{ minutos}$$

Búsqueda Adversarial

- Es una búsqueda en la cual examinamos el problema de planificar una acción en un dominio mientras otros agentes planifican contra nosotros
- O sea, hablamos de situaciones con más de un agente buscando una solución, en el mismo dominio (espacio de búsqueda).
- Esta situación ocurre habitualmente en juegos

Búsqueda Adversarial

- El ambiente con más de un agente se llama ambiente multiagente, en el cual cada agente es un oponente. En consecuencia, cada agente necesita considerar la acción de otros agentes y el efecto de esa acción en su propio desempeño.
- Esto es lo que se conoce como búsqueda adversarial, habitualmente conocida como juegos.
- Los juegos se modelan como problemas de búsqueda y funciones de evaluación heurística. Estos son los dos elementos que ayudan a modelar y resolver juegos en IA.

Tipos de juegos en IA

- Información perfecta: Aquella situación en la cual los agentes tienen una visión completa de un tablero. Los agentes tienen toda la información sobre el juego y pueden ver lo que cada otro agente hace.
- Ejemplos son el ajedrez, damas, Go, etc.
- Información imperfecta: Aquella situación en la cual no se tiene toda la información información sobre el juego y no se tiene certeza sobre lo que sucederá.
- Ejemplos son el juego del gato, batalla naval, bridge, entre otros.

Tipos de juegos en IA

- Determinísticos: Son aquellos que siguen un patrón estricto y un conjunto de reglas, donde no hay aleatoriedad.
- Ejemplos: Ajedrez, Damas, Go, Juego del gato, etc.
- No-determinísticos: Son aquellos en los cuales existen varios eventos impredecibles y existe un factor de aleatoriedad (suerte). Esta suerte está gatillada por medio de dados o cartas. either dice or cards. Cada respuesta no es fija. Se les llama también juegos estocásticos.
- Ejemplos: Backgammon, Monopolio, Póker, etc.

Árboles de juegos

- Un árbol de juego es un árbol en el cual los nodos son los estados del juego y las aristas del árbol son los movimientos que hacen los jugadores. El árbol de juegos contiene un estado inicial, funciones (acciones) y una función resultado.
- Ejemplo: El juego del gato.

Ahora vamos al minimax

- Es una regla de decisión usada en inteligencia artificial, teoría de decisiones, teoría de juegos, estadística y filosofía...
- Trata de minimizar las posibles pérdidas en el caso peor (un escenario de pérdidas máximas).
- Se puede resumir como la estrategia que consiste en elegir el mejor movimiento para ti, suponiendo que tu adversario elegirá el peor movimiento para ti.

Juegos para dos Personas

- En juegos con dos jugadores
 - llamados MIN y MAX
 - generalmente MAX mueve primero, luego se turnan
- MAX debe encontrar una estrategia para llegar a un estado ganador
 - no importa lo que haga MIN
- MIN hace lo mismo
 - o al menos trata de evitar que MAX gane

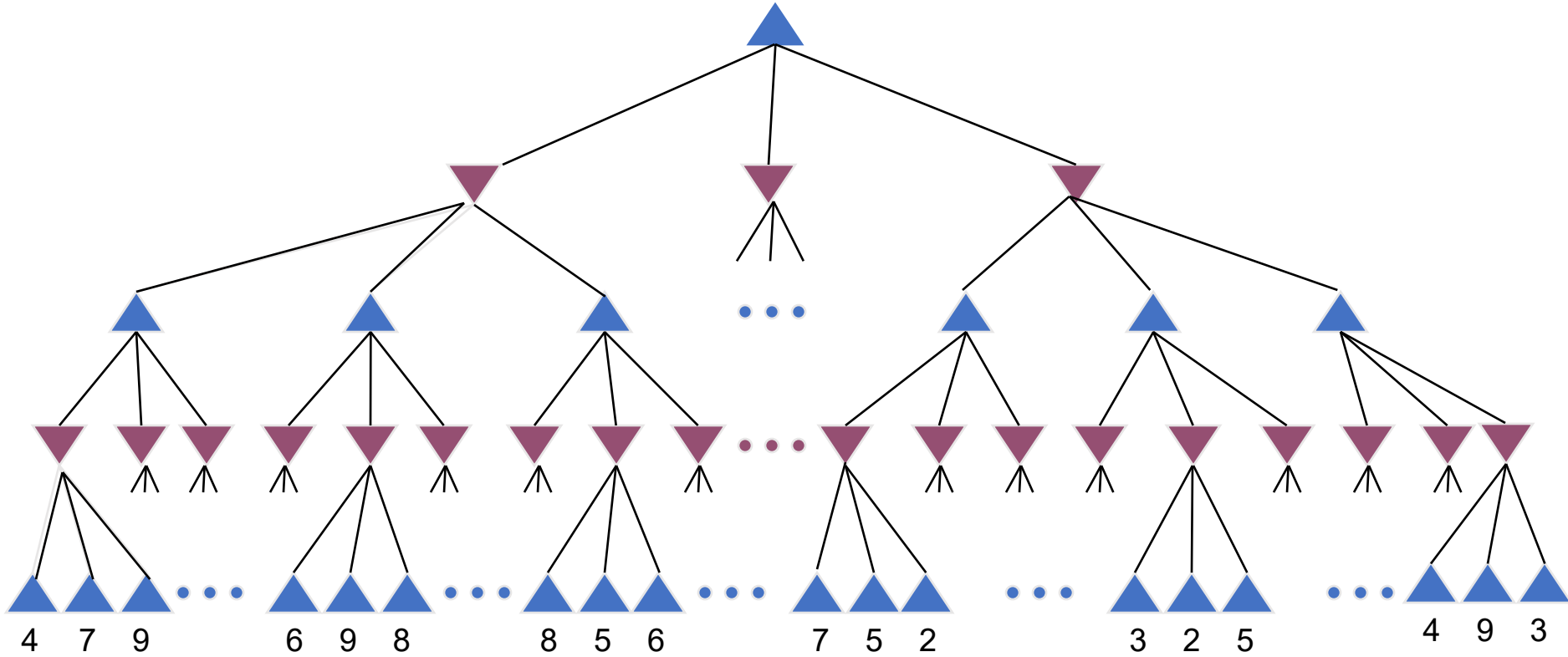
Decisiones Perfectas

- Basadas en una estrategia racional (optimal) para MAX
 - recorren todas las partes relevantes del árbol de búsqueda
 - esto debe incluir posibles movidas de MIN
 - identifican un camino que lleva a MAX a un estado ganador
- A veces no es muy práctico
 - limitaciones de tiempo y espacio

Estrategia Minimax

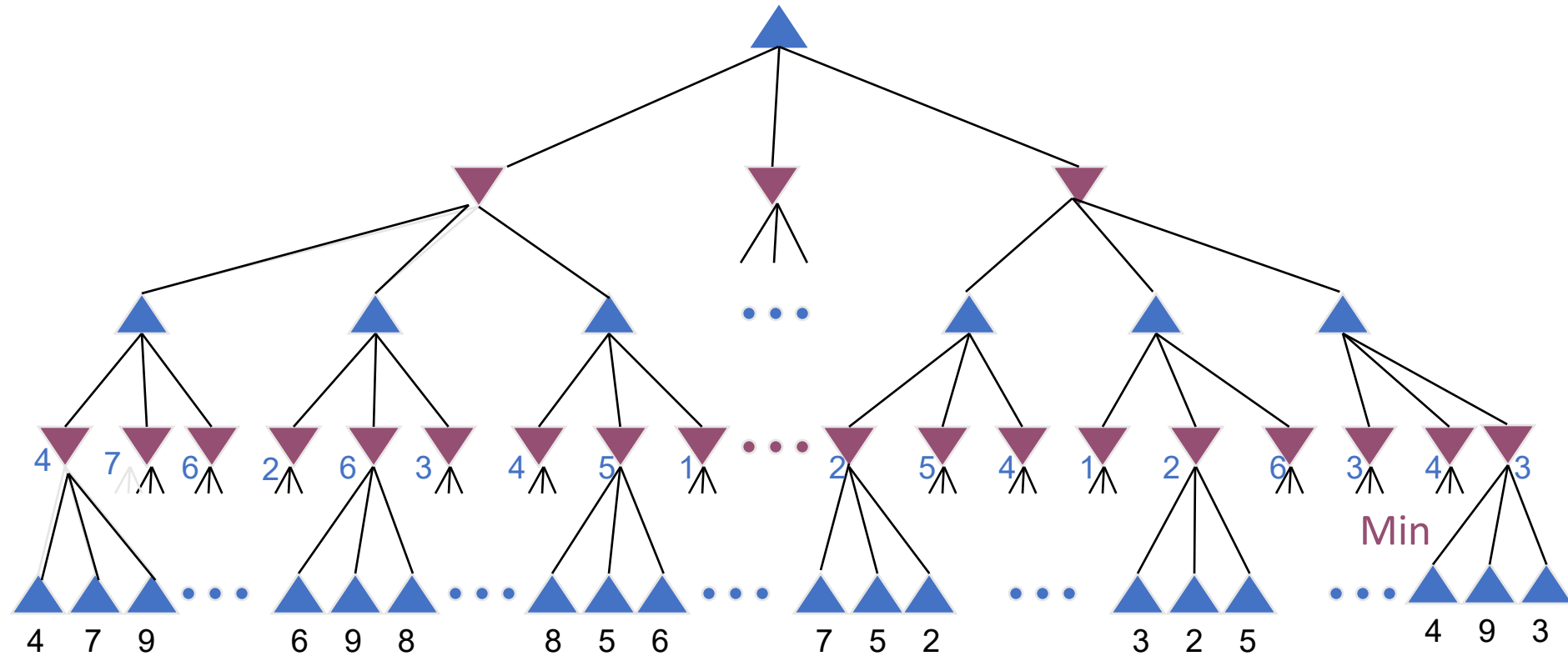
- Estrategia optimal para MAX:
 - Genera el árbol de juego
 - Calcula el valor de cada estado terminal
 - Basado en la función de utilidad
 - Calcula la utilidad de los nodos de mayor nivel
 - Partiendo desde las hojas hacia la raíz
 - MAX selecciona el nodo con valor más alto
 - MAX supone que MIN en su movida seleccionará el nodo con menor valor

Ejemplo Minimax



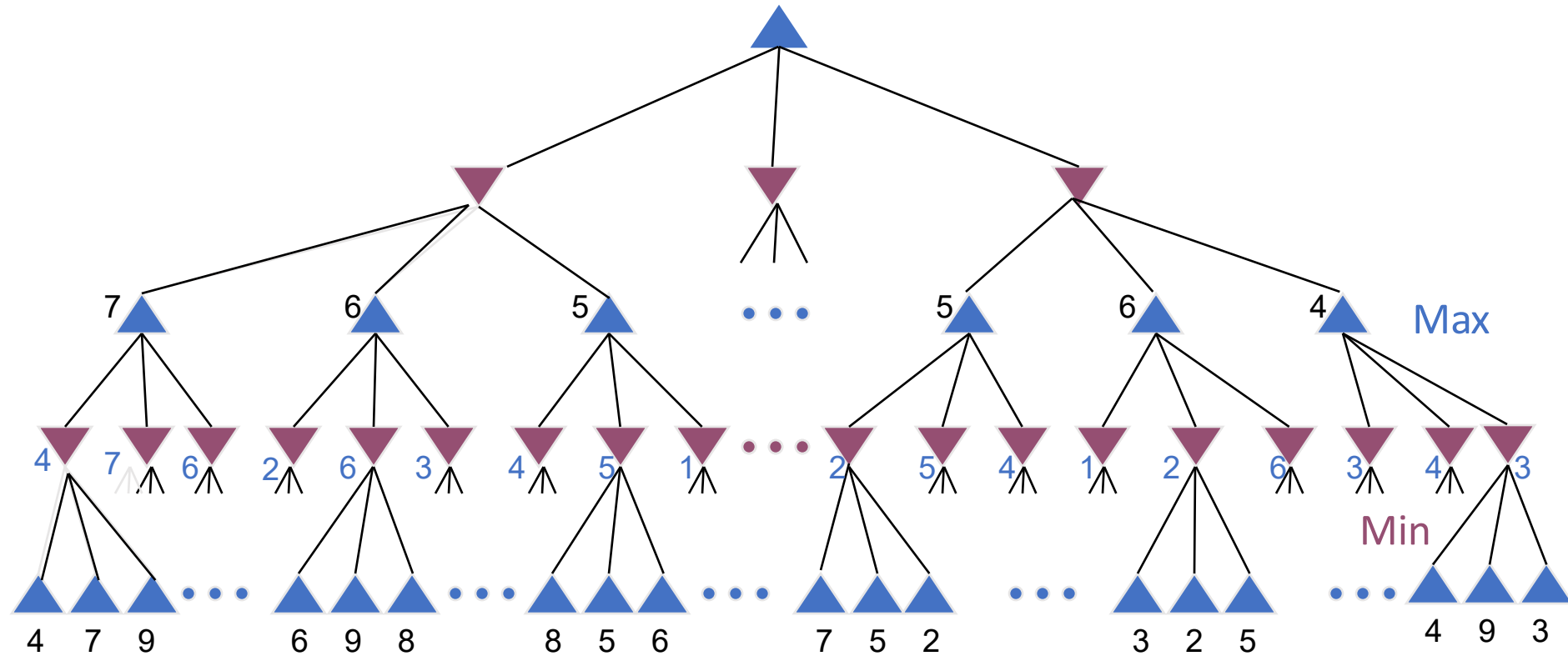
Nodos terminales: valores calculados a partir de la función de utilidad

Ejemplo Minimax

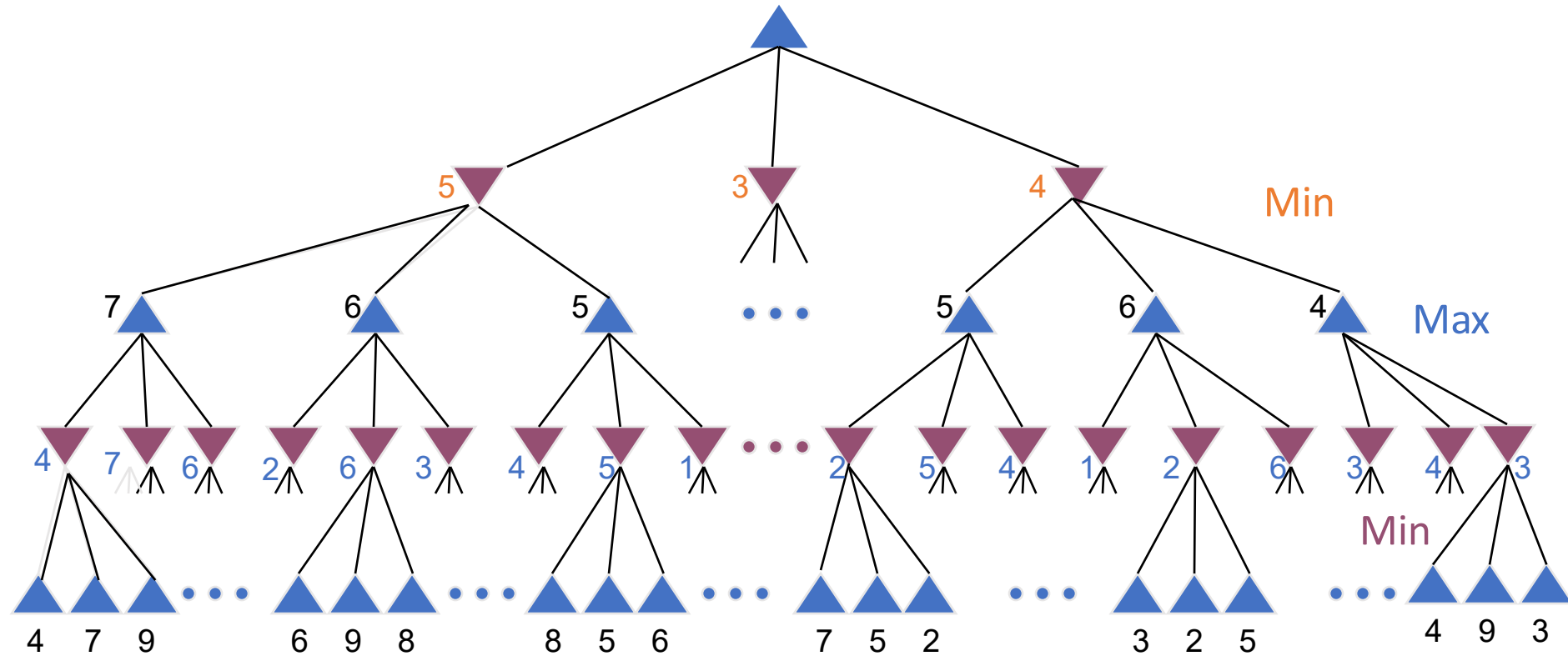


Otros nodos calculados con Minimax

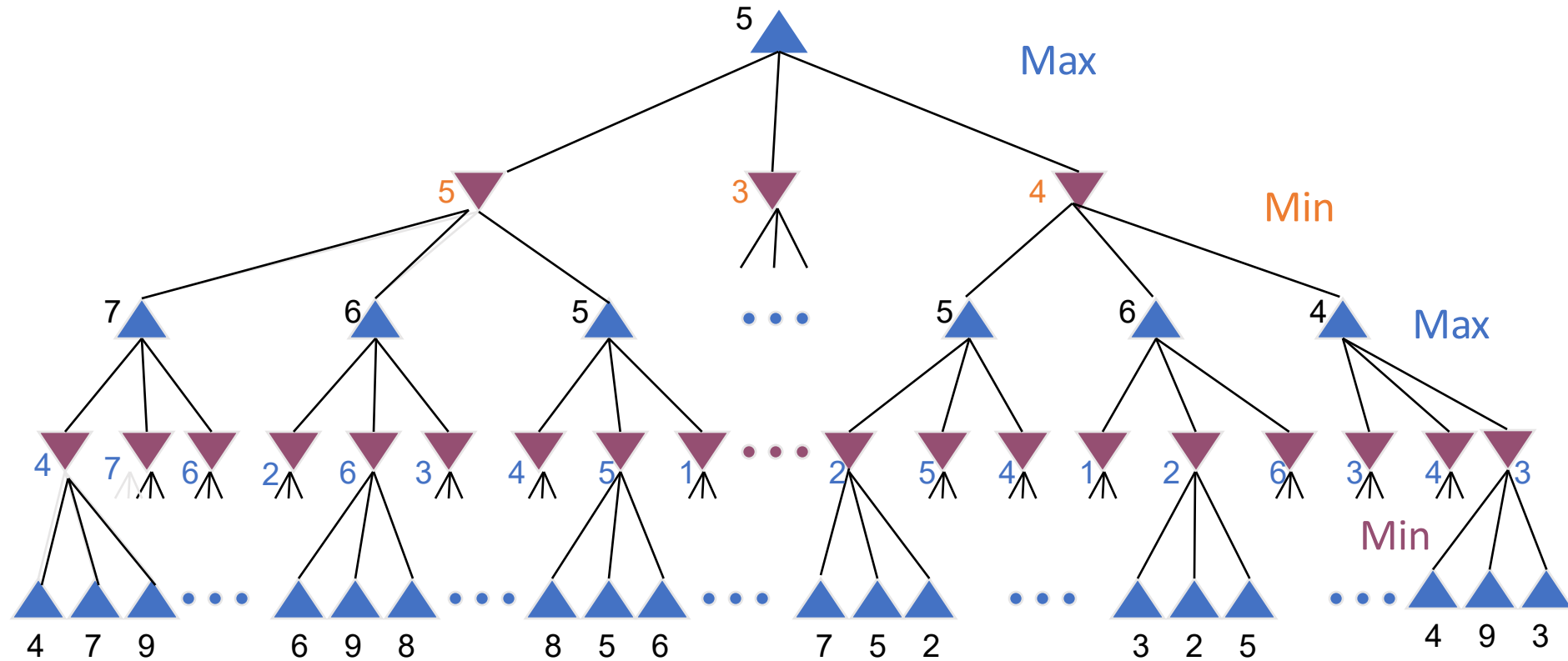
Ejemplo Minimax



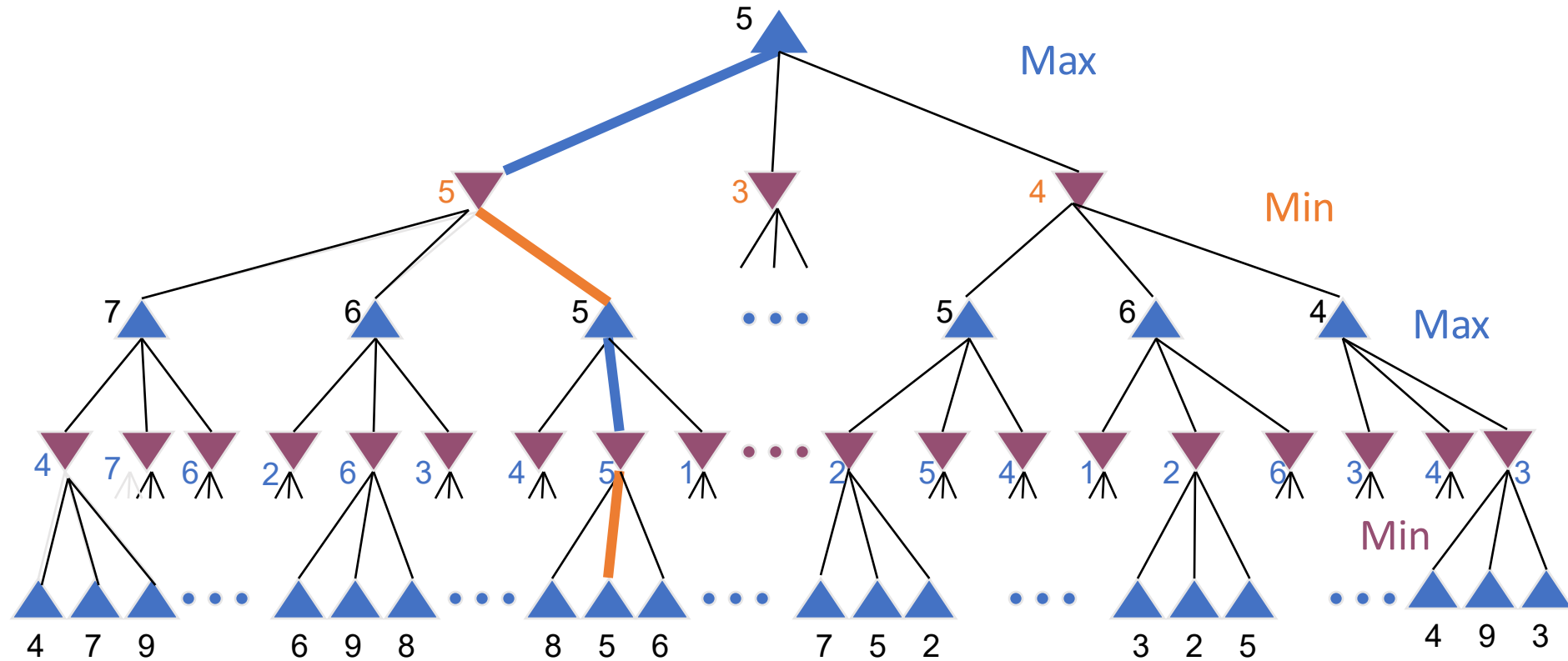
Ejemplo Minimax



Ejemplo Minimax



Ejemplo Minimax



Decisiones Imperfectas

- La búsqueda completa es impracticable para la mayoría de los juegos
- Alternativa:
 - buscar sólo parte del árbol
 - requiere un test de corte para determinar cuándo parar
 - usa una función de evaluación basada en heurísticas para estimar la utilidad esperada del juego a partir de una posición dada

Función de Evaluación

- Determina la eficiencia de un programa de juegos
- Debe ser consistente con la función de utilidad
- Compromiso entre precisión y costo en tiempo
 - sin límites de tiempo, puede usarse minimax

Función de Evaluación

- Debería reflejar las posibilidades reales de ganar
- Con frecuencia usa funciones lineales ponderadas

$$E = w_1f_1 + w_2f_2 + \dots + w_nf_n$$

combinación de características, ponderadas de acuerdo a su relevancia

El Problema del Horizonte

- Los movimientos realizados pueden tener consecuencias desastrosas en el futuro, pero éstas no son visibles ahora
 - los cambios correspondientes en la función de evaluación no serán evidentes hasta en niveles más profundos
 - Esos cambios están “detrás del horizonte”
- Determinar el horizonte es un problema abierto aún sin una solución general
 - sólo algunas aproximaciones pragmáticas restringidas a juegos o situaciones específicas

Minimax con el juego del gato

Cómo lo haría la IA

X	O	O
X		
O	X	

X	O	O
X		X
O	X	

X	O	O
X		
O	X	X

X	O	O
X	X	
O	X	

X	O	O
X	O	X
O	X	

X	O	O
X		X
O	X	O

X	O	O
X	O	
O	X	X

X	O	O
X		O
O	X	X

X	O	O
X	X	O
O	X	

X	O	O
X	X	
O	X	O

X	O	O
X	O	X
O	X	X

X	O	O
X	X	X
O	X	O

X	O	O
X	O	X
O	X	X

X	O	O
X	X	O
O	X	X

X	O	O
X	X	O
O	X	X

X	O	O
X	X	X
O	X	O

Y vamos a valorizar las posiciones

- Diremos que se tiene un valor cero, si es empate
- El valor -1 si ganan las O
- El valor 1 si ganan las X

X	O	O
X		
O	X	

1

X	O	O
X		
O	X	X

-1

X	O	O
X	X	
O	X	

1

X	O	O
X		X
O	X	

-1

X	O	O
X	O	X
O	X	

-1

X	O	O
X		X
O	X	O

1

X	O	O
X	O	
O	X	X

-1

X	O	O
X		O
O	X	X

1

X	O	O
X	X	O
O	X	

1

X	O	O
X	X	
O	X	O

1

X	O	O
X	O	X
O	X	X

-1

X	O	O
X	X	X
O	X	O

1

X	O	O
X	O	X
O	X	X

-1

X	O	O
X	X	O
O	X	X

1

X	O	O
X	X	O
O	X	X

1

X	O	O
X	X	X
O	X	O

1

Poda Alfa - Beta

- No es exactamente un nuevo algoritmo
- Es una técnica de optimización para el Minimax
- Permite reducir el tiempo de cómputo
- Corta aquellas ramas que no necesitan ser examinadas (porque ya existe un movimiento mejor)
- Alfa y Beta son dos parámetros
- se aplica a movimientos de ambos jugadores

Poda Alfa - Beta

- No es exactamente un nuevo algoritmo
- Es una técnica de optimización para el Minimax
- Permite reducir el tiempo de cómputo
- Corta aquellas ramas que no necesitan ser examinadas (porque ya existe un movimiento mejor)
- Alfa y Beta son dos parámetros
- Se aplica a movimientos de ambos jugadores
 - α indica la mejor elección para **Max**, así que nunca disminuirá
 - β indica la mejor elección para **Min**, así que nunca aumentará

Poda

- Descarta partes del árbol de búsqueda
 - aquellas que garantizadamente no serán buenos movimientos
 - aquellas en las que con una alta probabilidad no influirán en la solución
- Resulta un ahorro substancial tanto en tiempo como en espacio
 - sin embargo, la parte de la tarea que queda puede ser exponencial

Poda Alfa-Beta

- Movimientos que con certeza no tendrán una buena evaluación **no son considerados** (son podados)
 - ellos llevarían a resultados menos deseables
- se aplica a movimientos de ambos jugadores
 - α indica la mejor elección para **Max, así que nunca disminuirá**
 - β indica la mejor elección para **Min, así que nunca aumentará**

- Alfa – Beta es un algoritmo de búsqueda en profundidad
- Alfa y Beta son dos variables que debemos tener en cuenta en el recorrido del árbol
- Alfa será el máximo encontrado hasta ese momento
- Beta será el mínimo encontrado hasta ese momento

- En el ejemplo, los valores estarán en el rango de 1 a 9
- En los movimientos del contrincante, suponemos que escogerá los elementos que minimizan nuestra utilidad
- En nuestros movimientos supondremos que haremos los movimientos que maximicen nuestra utilidad

Poda alfa - beta

- Esta mejora consiste en mantener un intervalo de las mejores jugadas encontradas
- Tanto para el jugador Alfa como para el jugador Beta, cuyos valores iniciales son $(-\infty, +\infty)$. Estos valores se van actualizando a medida que recorremos el árbol
- En nuestros movimientos supondremos que haremos los movimientos que maximicen nuestra utilidad

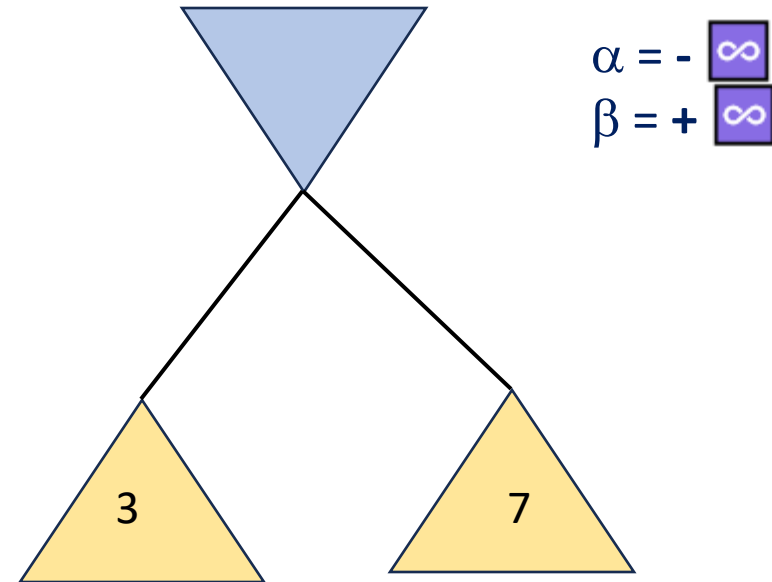
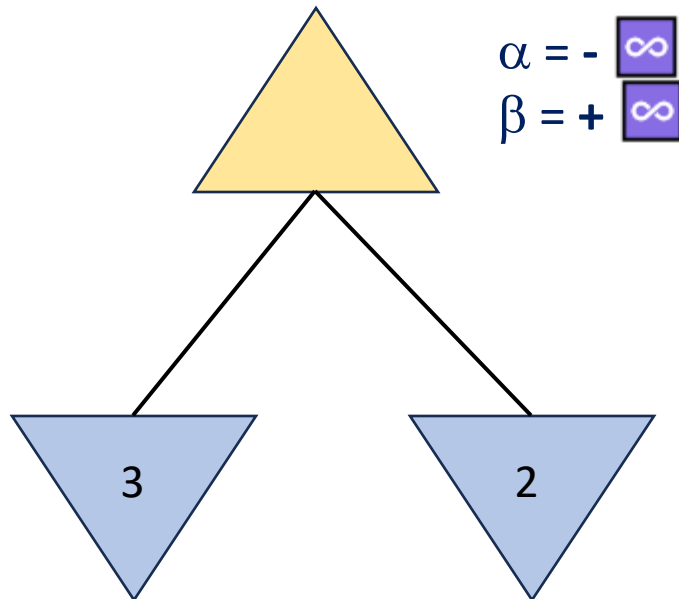
Poda alfa – beta (Cont.)

- Para el jugador Alfa y para el jugador Beta, los valores iniciales son ($-\infty$, $+\infty$).
- Estos valores se van actualizando a medida que recorremos el árbol
- En nuestros movimientos supondremos que haremos los movimientos que maximicen nuestra utilidad

Poda alfa – beta (Cont.)

Cuando para un nodo se obtiene el valor de unos de sus hijos, este valor se actualiza

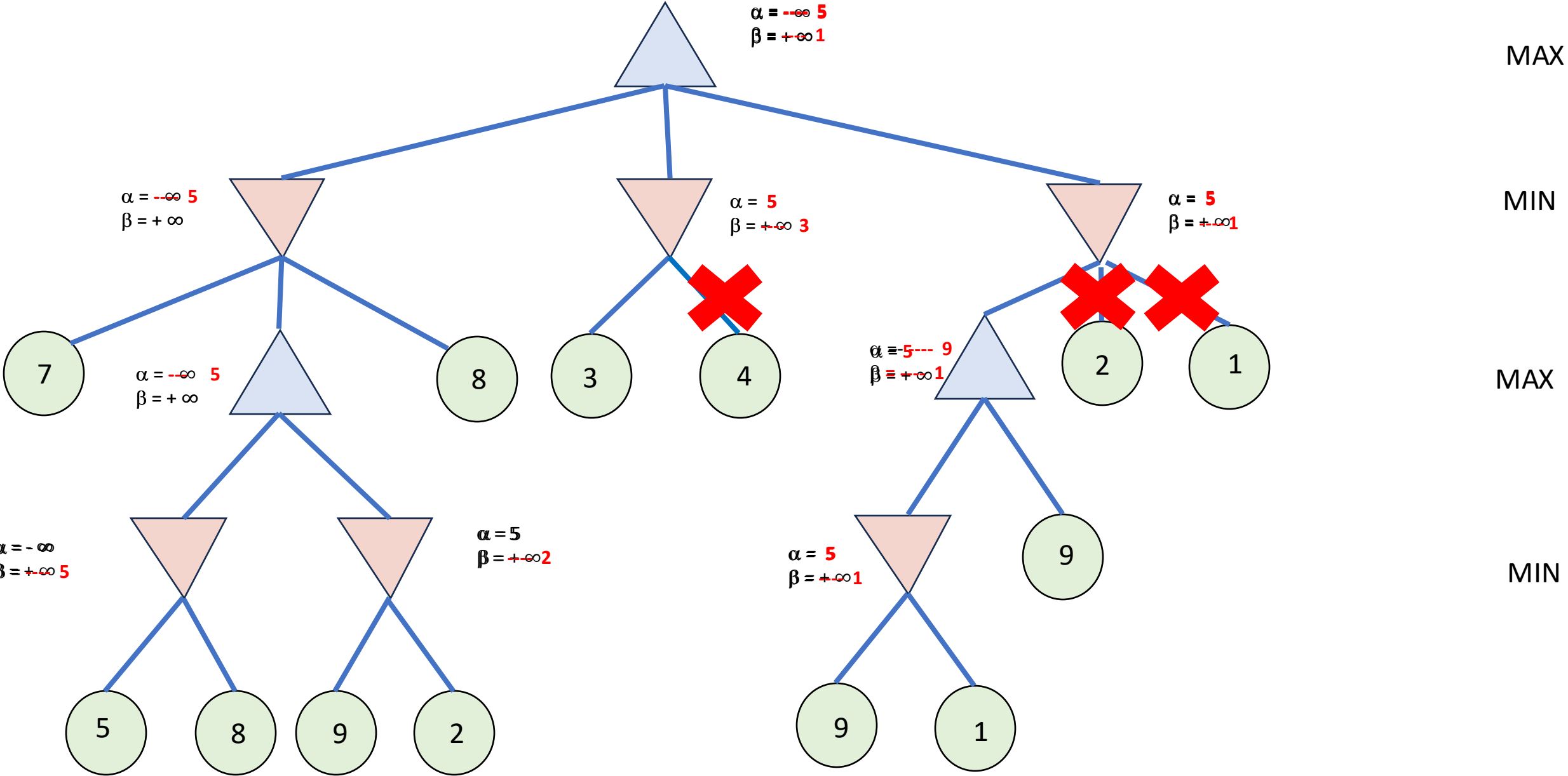
Si es un nodo MAX, se actualiza el valor de alfa, si el nuevo valor es mayor, si es un nodo MIN, se actualiza el valor de beta, si el nuevo valor es menor



Poda alfa – beta (Cont.)

Si el valor de alfa es mayor o igual a beta, significa que el valor para el nodo no puede ser mejorado, y por lo tanto no es necesario seguir evaluando el resto de las ramas.

Ejemplo



Procesos de Markov

- Las cadenas de Markov y los procesos de Markov son un tipo especial de procesos estocásticos que poseen la siguiente propiedad:
 - Conocido el estado del proceso en un momento dado, su comportamiento futuro no depende del pasado.
 - Dicho de otro modo, “dado el presente, el futuro es independiente del pasado”

Cadenas de Markov

- Las cadenas de Markov son modelos probabilísticos que se usan para predecir la evolución y el comportamiento a corto y a largo plazo de determinados sistemas.
- Ejemplos: reparto del mercado entre marcas; dinámica de las averías de máquinas para decidir política de mantenimiento; evolución de una enfermedad,...

Definición

- Una **Cadena de Markov** (CM) es:
 - Un proceso estocástico
 - Con un número finito de estados (E)
 - Con probabilidades de transición estacionarias
 - Que tiene la propiedad markoviana

Transiciones

- Entonces, existen estados $E_1, E_2, E_3, \dots$ de manera que se pasa de uno a otro por una matriz de transición en una etapa.
- La cardinalidad (número de elementos) del conjunto de estados es numerable, es decir, es un conjunto finito o con la misma cardinalidad que los números naturales.

La matriz de transición

- La matriz de transición en cada etapa tiene como elementos a las probabilidades de paso de un estado a otro cuando el proceso evoluciona desde una etapa n a la etapa siguiente $n+1$.
- Por lo tanto, está compuesta de números reales positivos entre 0 y 1, de manera que la suma de cada fila o columna, según la disposición de los estados inicial (en la etapa n) y final (en la etapa $n+1$), es 1.

Aplicación en medicina

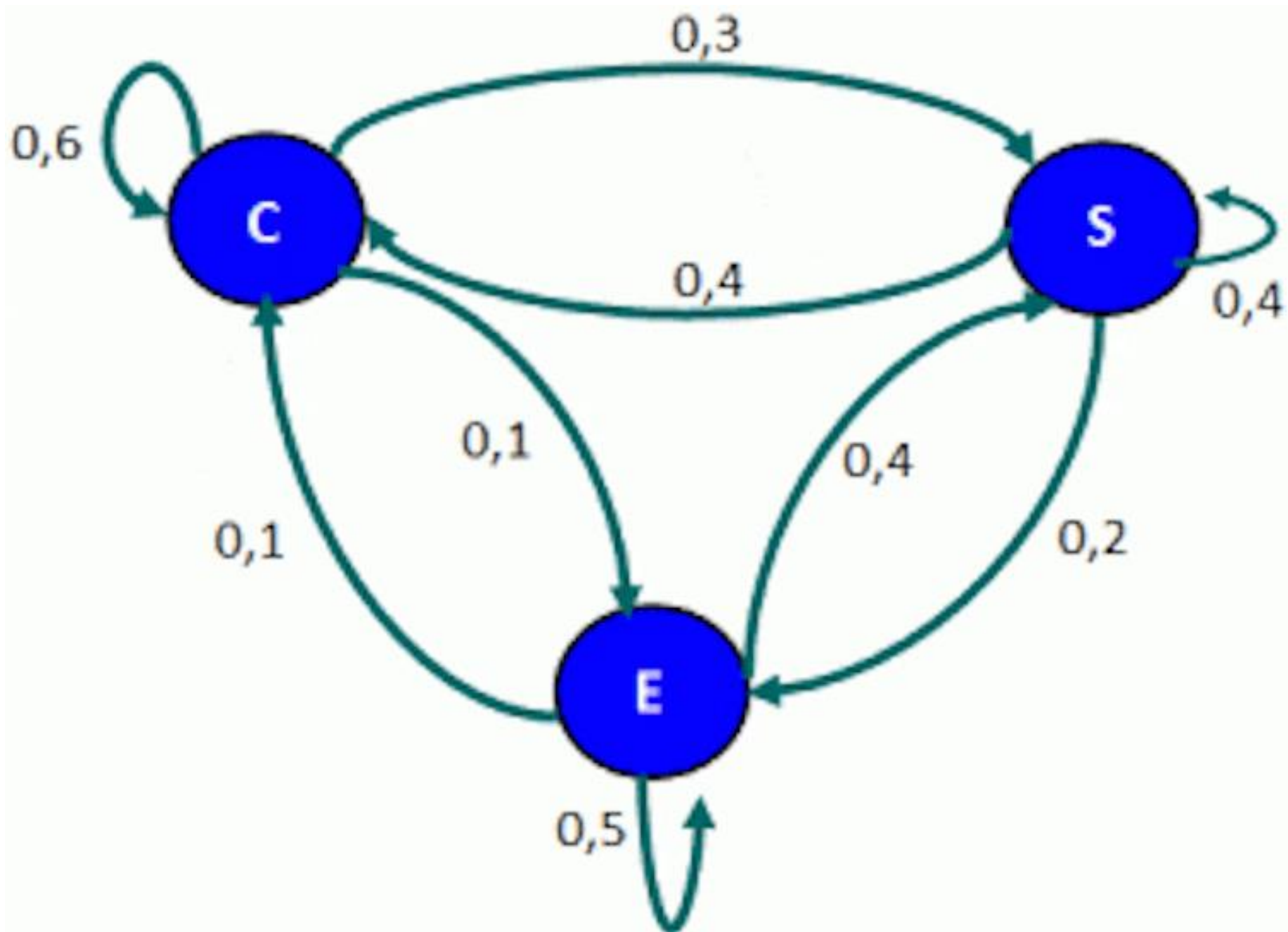
- En una unidad de cuidados intensivos, los pacientes se clasifican (*triage*) atendiendo a su estado: crítico, serio y estable.
- Cada día se actualizan las clasificaciones de acuerdo con la evolución histórica de los pacientes admitidos en la unidad hasta ese momento, de modo que las frecuencias relativas de cambios de estado de un paciente son:

Cambios de estado

	Crítico	Serio	Estable
Crítico	0,6	0,3	0,1
Serio	0,4	0,4	0,2
Estable	0,1	0,4	0,5

- En la figura, las entradas por filas están asociadas al estado del paciente en el día n y las columnas se refieren a su estado en el día $n+1$. Entonces, podríamos tomar como matriz de transición en una etapa:

0,6	0,3	0,1
0,4	0,4	0,2
0,1	0,4	0,5



- Con un gráfico como este podemos calcular probabilidades en más de una etapa. Por ejemplo, la probabilidad de pasar del estado crítico C a estable E en dos días. Hay tres posibles caminos, dependiendo del estado C, S y E del paciente después del primer día:

- $C \rightarrow C \rightarrow E$
- $C \rightarrow S \rightarrow E$
- $C \rightarrow E \rightarrow E$

- Solo tenemos que multiplicar las probabilidades y sumar de la forma:
- $0,6 \times 0,1 + 0,3 \times 0,2 + 0,1 \times 0,5 = 0,17$
- Es decir, un paciente ingresado en estado crítico C evolucionará al estado estable E en dos días en un 17 % de las ocasiones. Evidentemente, estas probabilidades pueden cambiar conforme aparecen nuevos tratamientos, o podrían definirse distintos estados de la cadena atendiendo a la edad del paciente, etc. Todas estas generalizaciones enriquecerían la cadena de Márkov, pero la idea general seguiría siendo la misma.

Procesos estocásticos. Ejemplos

- Serie mensual de ventas de un producto
- Estado de una máquina al final de cada semana (funciona/averiada)
- Número de clientes esperando en una cola cada 30 segundos
- Marca de detergente que compra un consumidor cada vez que hace la compra. Se supone que existen 7 marcas diferentes
- Número de unidades en bodega al finalizar la semana

Ejemplo de problemas (1)

- Para efectos de una investigación, en un determinado país, una familia puede clasificarse como habitante de zona urbana, rural o suburbana. Se ha estimado que durante un año cualquiera, el 15% de todas las familias urbanas se cambian a zona suburbana y el 5% a zona rural. El 6% de las familias suburbanas pasan a zona urbana y el 4% a zona rural. El 4% de las familias rurales pasan a zona urbana y el 6% a zona suburbana.
- ¿Cómo se distribuirá la población el próximo año?

Ejemplo de problemas (2)

- El departamento de estudios de mercado de una fábrica estima que el 20% de la gente que compra un producto un mes, no lo comprará el mes siguiente. Además, el 30% de quienes no lo compran un mes lo adquirirá al mes siguiente. En una población de 1000 individuos, 100 compraron el producto el primer mes.
- ¿Cuántos lo comprarán al mes próximo?
- ¿Y dentro de dos meses?

Optimalidad de Bellman

- 1953, Richard Bellman inventa la programación dinámica
- Se utiliza para optimizar problemas complejos que pueden ser discretizados y secuencializados
- Una subestructura óptima significa que se pueden utilizar soluciones óptimas de subproblemas para encontrar la solución óptima del problema en su conjunto.

... Bellman

- En general, se pueden resolver problemas con subestructuras óptimas de la siguiente forma:
 - Dividir el problema en subproblemas más pequeños
 - Resolver estos problemas de manera óptima usando estos tres pasos recursivamente
 - Usar estas soluciones óptimas para construir una solución óptima al problema original

Próxima clase, lunes 30

- Método de Montecarlo
- Aprendizaje con refuerzo
- Algoritmos basados en poblaciones
- Algoritmos adaptativos y aprendizaje